

# Blockchain-Based Mutual Authentication Protocol for IoT-Enabled Decentralized Healthcare Environment

Chien-Ming Chen<sup>1</sup>, Senior Member, IEEE, Zhaoting Chen, Saru Kumari<sup>2</sup>, Senior Member, IEEE, Mohammad S. Obaidat<sup>3</sup>, Life Fellow, IEEE, Joel J. P. C. Rodrigues<sup>4</sup>, Fellow, IEEE, and Muhammad Khurram Khan<sup>5</sup>, Senior Member, IEEE

**Abstract**—In the ever-evolving landscape of technology, healthcare continuously harnesses its benefits, propelling advancements in medical practices. Within intelligent healthcare, medical robots play a pivotal role, providing integral support to healthcare professionals, streamlining processes, and delivering efficient services. These robots securely transmit patient treatment plans, transferring them to cloud storage and subsequently storing them in blockchain systems. This innovative approach ensures the integrity and accessibility of patient data, introducing novel avenues for seamless interaction with medical information for hospitals and patients' families. Despite these advantages, the looming privacy risks associated with sensitive patient data transmission pose a compelling challenge, demanding a comprehensive solution. In response to this challenge, we propose a mutual authentication and key agreement protocol designed to optimize healthcare services while prioritizing data security and patient privacy. To validate the robustness of our authentication protocol, we conduct thorough analyses based on both formal and informal models, establishing a foundational framework for evaluating the protocol's security. Additionally, we perform a comprehensive comparative analysis, assessing the proposed protocol against existing counterparts across various dimensions. This comparative scrutiny reveals the superiority of our protocol in terms of security, as well as its efficiency in communication cost

and computational overhead. These findings affirm the efficacy of our proposed solution in navigating the intricate interplay between medical robotics, blockchain, and data security.

**Index Terms**—Authentication, blockchain, healthcare, Internet of Things (IoT).

## I. INTRODUCTION

IN THE field of information technology, several prominent keywords have emerged, such as Internet of Things (IoT) [1], [2], [3], artificial intelligence, wearable devices, blockchain, etc. Among these, the IoT stands out as a foundational and prevalent technology that enables the interconnection and communication between various devices and the Internet. This facilitates intelligent information exchange and data sharing, with extensive applications in connected vehicles, environmental monitoring, smart cities, smart homes, healthcare, agriculture, and industrial IoT. These applications hold tremendous potential for enhancing efficiency, improving lives, and driving innovation.

Wearable technologies [4], [5], [6] refer to electronic devices and sensor technologies that can be worn on the body to interact with users, particularly significant in healthcare. It monitors patients' physiological parameters, behavioral activities, sleep quality, and other indicators, collecting personal data through built-in sensors. By providing comprehensive and objective patient data, wearable technology enables doctors to understand the patient's health condition and formulate more precise treatment plans. Integrating wearable technology with the healthcare industry offers efficient, convenient, and intelligent personalized healthcare services, enhancing medical efficiency and improving patient experiences [7], [8], [9].

When seeking medical treatment, patients select doctors and hospitals based on their circumstances, such as needing specialized treatment or immediate medical attention in emergencies. Therefore, flexible storage of patients' health data becomes essential [10], [11], [12]. Personal health data includes physiological parameters, exercise data, sleep quality reports, environmental information, and more. Dealing with such complex data sets requires sufficient storage space and computing capabilities. Traditional data processing and storage methods no longer meet these requirements, but cloud computing can address them by providing computing resources and related services. Cloud servers, operating as virtual servers in a cloud computing environment, efficiently store and process

Manuscript received 27 August 2023; revised 27 December 2023 and 28 February 2024; accepted 26 April 2024. Date of publication 13 May 2024; date of current version 9 July 2024. The work of Chien-Ming Chen was supported in part by the Startup Foundation for Introducing Talent of NUIST. The work of Muhammad Khurram Khan was supported by King Saud University, Riyadh, Saudi Arabia, under Project RSP2024R12. The work of Joel J. P. C. Rodrigues was supported in part by the Brazilian National Council for Scientific and Technological Development—CNPq under Grant 306607/2023-9. (Corresponding author: Saru Kumari.)

Chien-Ming Chen is with the School of Artificial Intelligence (School of Future Technology), Nanjing University of Information Science and Technology, Nanjing 211544, China (e-mail: chienmingchen@ieee.org).

Zhaoting Chen is with the College of Computer Science and Engineering, Shandong University of Science and Technology, Qingdao 266590, China (e-mail: sdkjdxchen@163.com).

Saru Kumari is with the Department of Mathematics, Chaudhary Charan Singh University, Meerut 250004, India (e-mail: saryusirohi@gmail.com).

Mohammad S. Obaidat is with the King Abdullah II School of Information Technology, The University of Jordan, Amman 11942, Jordan, also with the School of Computer and Communication Engineering, University of Science and Technology Beijing, Beijing 100083, China, also with the Department of Computational Intelligence, School of Computing, SRM University, Kattankulathur 603203, India, and also with the School of Engineering, The Amity University, Noida 201301, India (e-mail: m.s.obaidat@ieee.org).

Joel J. P. C. Rodrigues is with Amazonas State University, Manaus 69050-020, Brazil, and also with Lusófona University, 1749-024 Lisbon, Portugal (e-mail: joeljr@ieee.org).

Muhammad Khurram Khan is with King Saud University, Riyadh 11653, Saudi Arabia (e-mail: mkhurram@ksu.edu.sa).

Digital Object Identifier 10.1109/IIOT.2024.3396488

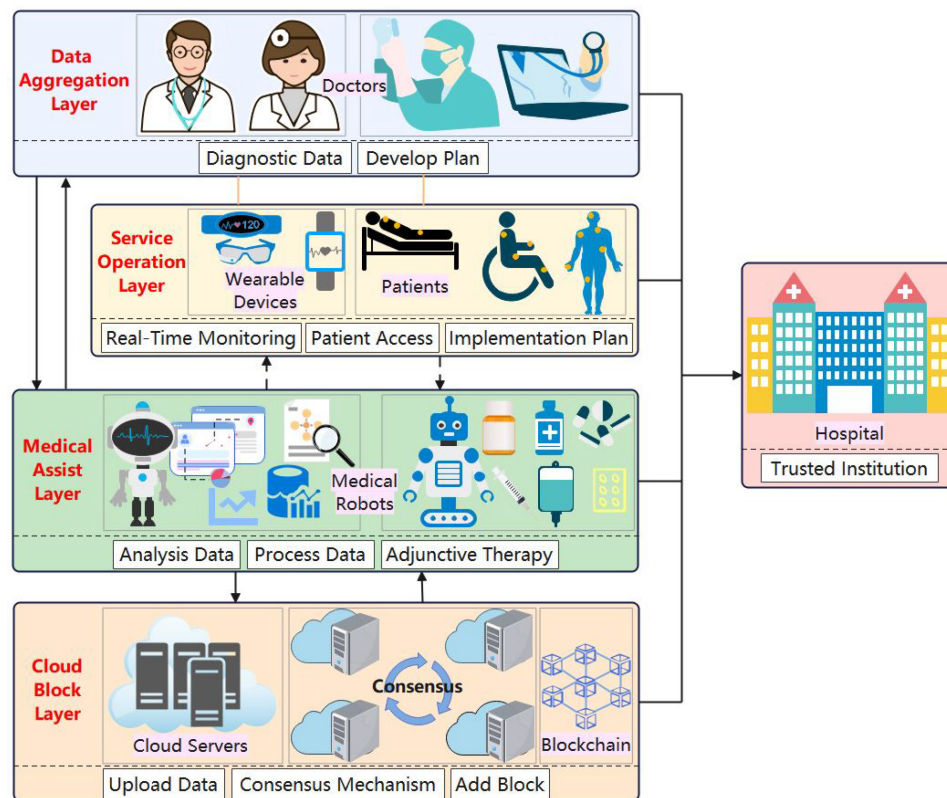


Fig. 1. Functional architecture.

data. To meet the demands of patient care and personal health data storage, wearable devices transmit the collected health data to cloud servers. However, the centralized architecture of cloud servers poses risks associated with centralized data storage, making them targets for hackers. To mitigate this security risk, blockchain offers enhanced data security and privacy protection [13], [14].

Blockchain is a decentralized distributed database where data is stored on multiple nodes within the network, employing consensus mechanisms to achieve agreement [15], [16]. Data storage and transaction operations in a blockchain are publicly visible, ensuring data transparency [17]. Cloud servers can function as blockchain nodes, participating in consensus and transaction verification while providing elastic computing and storage resources. This enhances data security, integrity, and trustworthiness. By combining cloud servers with blockchain technology, the advantages of cloud computing and blockchain can revolutionize the healthcare industry, creating new opportunities. For instance, wearable devices collect personal health data from patients and transmit it to cloud servers in real time for storage and processing. Simultaneously, the cloud servers act as blockchain nodes, storing personal data related to patient health on the blockchain. Doctors can analyze this data to promptly diagnose and treat patients and provide medical advice. The patient's family members can access and review this data to comprehensively understand the patient's current health status.

The scarcity of healthcare personnel is a pressing issue that necessitates immediate attention. As a result, the utilization of medical robots [18] has emerged as a potential

solution. Medical robots can perform specific medical tasks and assist healthcare professionals in various capacities. For example, in the case of hospitalized patients, who often require more intensive care due to the severity of their conditions, medical professionals face challenges in providing meticulous attention, primarily due to their busy schedules and the need to attend to multiple patients simultaneously. In such scenarios, medical robots can play a significant role.

Fig. 1 illustrates the operational framework of medical robots. Doctors furnish medical robots with essential information, including the list of patients requiring treatment, ward numbers, medication times, drug dosages, and drug ratios. Subsequently, the medical robots retrieve the necessary medication based on the provided drug information and identify the corresponding patients using known patient data. Only after confirming the patients' identities can the medical robots administer medication or perform injections. Furthermore, medical robots generate encrypted transactions in partial blocks, encompassing patient information, medication details, injection information, and other relevant data. These partial blocks are then uploaded to the hospital's cloud storage, where they can be accessed by the hospital staff and the patient's families. The cloud server creates complete blocks after successfully verifying the uploaded partial blocks. Finally, multiple cloud servers employ a consensus algorithm to determine whether the complete block should be added to the blockchain. Implementing medical robots offers several advantages, including enhanced efficiency for healthcare professionals, alleviating their workload, assurance of accuracy

and safety throughout the medical processes, and providing higher quality medical services to patients.

One of the primary obstacles currently faced by the healthcare industry pertains to data security, as medical data encompasses patient privacy and life safety. To safeguard this sensitive data, the most efficient mechanism involves implementing identity authentication and data encryption during data transmission. Identity authentication ensures the legitimacy and authenticity of the communicating parties, effectively mitigating the risk of data leakage. Data encryption utilizes a shared session key (SK) to encrypt the transmitted data, thereby ensuring the security of private information. The ultimate objective of securely storing and transmitting personal health data is to protect patients' well-being and ensure that doctors can access accurate patient data promptly, facilitating timely and effective treatment. To fulfill this purpose, we have devised a robust authentication protocol that optimizes healthcare services.

The main contributions of this work are listed in the following.

- 1) To ensure the security and confidentiality of patient treatment protocols in the medical field, we propose a mutual authentication protocol involving doctors, medical robots, cloud servers, and patients. In addition, we use biometric information to improve the protocol's security further.
- 2) A formal security analysis is performed with the real-or-random (ROR) model to verify its security. We also demonstrate that our work satisfies various security requirements.
- 3) Our work is compared with competing existing protocols and proves that the protocol has good advantages in terms of security and performance. The comparison results further confirm the reliability of the protocol.

The remainder of this article is structured as follows. Section II reviews the related work. Section III describes the preliminaries. In Section IV, we describe the proposed protocol. Sections V and VI perform security analysis and performance comparisons. Finally, we conclude the work of this article.

## II. RELATED WORK

Numerous studies have been conducted on secure authentication protocols in healthcare environments. Wu et al. [19] proposed a smart card-based telemedicine protocol aimed at enhancing the performance and efficiency of protocols in telemedicine environments. However, their protocol [19] is susceptible to impersonation attacks and privileged insider attacks, as highlighted by Debiao et al. [20], who proposed an improved protocol to address these vulnerabilities. Despite Debiao et al. [20] claiming that the enhanced protocol is secure, Wei et al. [21] argued that their protocol [20] is still vulnerable to offline password-guessing attacks. Similarly, Zhu [22] identified the same security concerns regarding Wei et al.'s protocol [21] and proposed a new medical authentication protocol. Considering the necessity of providing healthcare services to a broader user base,

Jia et al. [23] put forward a bilinear pairing-based fog-driven three-party protocol for healthcare IoT. Nevertheless, Wu et al. [24] and Chen et al. [25] argued that Jia et al.'s protocol [23] fails to prevent attacks with known session-specific temporary information and cannot ensure user anonymity and untraceability. Furthermore, Shamshad et al. [26] highlight the protocol's inability to resist impersonation attacks.

Medical data is closely associated with patient health, and cloud computing is gaining popularity as it offers more secure data sharing and storage capabilities. In 2014, Chen et al. [27] proposed a privacy authentication and key agreement protocol in a cloud environment, leveraging the characteristics of mobile devices. However, Chiou et al. [28] pointed out that Chen et al.'s [27] protocol cannot guarantee patient anonymity and propose a more robust protocol tailored to the medical environment. Mohit et al. [29] further discovered that Chiou et al.'s [28] protocol is vulnerable to mobile device theft attacks, prompting them to propose an authentication protocol for healthcare systems based on cloud computing technology. Jiang et al. [30] improved Wu et al.'s [31] protocol and introduced a health cloud three-factor protocol that utilizes a fuzzy extractor to safeguard biometric privacy. However, Chen and Chen [32] reviewed Jiang et al.'s protocol [30] and indicated its insecurity, susceptibility to replay attacks, denial-of-service attacks, and attacks involving known session-specific temporary information. With the advent of wearable technology, Kumar et al. [33] proposed cloud-assisted efficient protocols for wireless body area networks. At the same time, Alzahrani [34] suggested a secure authentication protocol for cloud-based wireless sensor networks.

Blockchain technology, functioning as a distributed database, demonstrates proficiency in storing and sharing healthcare data. Garg et al. [35] devised an inventive healthcare authentication protocol incorporating blockchain technology to establish a more secure key agreement protocol. In 2021, Wang et al. [36] concentrated on the application of wireless medical sensor networks in healthcare. They proposed a lightweight authentication protocol employing blockchain technology and physically unclonable functions (PUFs) to augment the protocol's security. However, Yu and Park [37] contended that Wang et al.'s protocol [36] lacks effective mutual authentication and is susceptible to man-in-the-middle attacks and SK leakage attacks. In a different vein, Xiong et al. [38] delved into blockchain applications in the context of Bitcoin. They proposed a novel batch authentication protocol grounded in an elliptic curve digital signature algorithm in blockchain healthcare IoT. Additionally, they introduced a group testing technique for efficiently identifying invalid signatures. In 2022, Wazid et al. [39] thoroughly considered the healthcare sector's risk of ransomware and proposed a security architecture that effectively mitigates ransomware attacks by integrating blockchain technology. Xiang et al. [40] provided a decentralized blockchain-based authentication and access control protocol to ensure healthcare data security. Other researchers have also explored the application of blockchain and cloud computing technologies in healthcare. Son et al. [41] employed cipher-policy attribute-based encryption and blockchain technology to manage a large volume of patient data stored on cloud servers, thus ensuring data integrity. Additionally,

TABLE I  
SUMMARY OF AUTHENTICATION PROTOCOLS

| Protocols          | Methods                                                                                                                                         | Limitations/Challenges                                                                                                                                          |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Wu et al. [19]     | (1)Use symmetric cryptography<br>(2)Use discrete logarithm problem                                                                              | (1)Unable to resist impersonation attacks<br>(2)Unable to resist privileged insider attacks                                                                     |
| Debiao et al. [20] | (1)Use exponential operation<br>(2)Use modular inversion operation<br>(3)Use addition operation                                                 | (1)Unable to resist offline password-guessing attacks                                                                                                           |
| Wei et al. [21]    | (1)Use exponential operation<br>(2)Use modular inversion operation                                                                              | (1)Unable to resist offline password-guessing attacks                                                                                                           |
| Jia et al. [23]    | (1)Use bilinear pairing<br>(2)Use scala multiplication                                                                                          | (1)Unable to resist known session-specific temporary information attacks<br>(2)Unable to provide user anonymity and untraceability                              |
| Chen et al. [27]   | (1)Use bilinear pairing<br>(2)Use elliptic curve digital signature algorithm<br>(3)Use symmetric cryptography<br>(4)Use asymmetric cryptography | (1)Unable to provide user anonymity                                                                                                                             |
| Chiou et al. [28]  | (1)Use symmetric cryptography<br>(2)Use elliptic curve digital signature algorithm<br>(3)Use bilinear pairing                                   | (1)Unable to resist stolen smart device attacks                                                                                                                 |
| Jiang et al. [30]  | (1)Use symmetric cryptography<br>(2)Use scalar multiplication<br>(3)Use fuzzy extractor                                                         | (1)Unable to resist denial-of-service attacks<br>(2)Unable to resist known session-specific temporary information attacks<br>(3)Unable to resist replay attacks |
| Wu et al. [31]     | (1)Use symmetric cryptography<br>(2)Use scalar multiplication<br>(3)Use fuzzy extractor                                                         | (1)Unable to resist impersonation attacks<br>(2)Unable to resist offline password-guessing attacks                                                              |
| Wang et al. [36]   | (1)Use blockchain<br>(2)Use PUF<br>(3)Use fuzzy extractor                                                                                       | (1)Unable to resist man-in-the-middle attacks<br>(2)Unable to resist session key leakage attacks<br>(3)Unable to provide effective mutual authentication        |
| Xiong et al. [40]  | (1)Use blockchain<br>(2)Use elliptic curve digital signature algorithm<br>(3)Use addition operation                                             | (1)Unable to provide user anonymity                                                                                                                             |
| Zhang et al. [43]  | (1)Use blockchain<br>(2)Use symmetric cryptography<br>(3)Use attribute-based encryption                                                         | (1)Some economic aspects in its implementation                                                                                                                  |

Itoo et al. [42] incorporated an elliptic curve encryption algorithm and blockchain technology to enhance the security of cloud-assisted healthcare key agreement in their protocol. For achieving fine-grained access control and privilege revocation in healthcare data, Zhang et al. [43] introduced a blockchain-based data-sharing protocol in healthcare environments. This protocol combines traditional cloud servers with distributed blockchain, utilizing symmetric searchable encryption and attribute-based encryption to achieve fine-grained data access control. The authors update the most recent set of attributes in the private blockchain to verify identity and decide whether to grant or revoke relevant permissions. However, this approach encountered economic challenges in its implementation [44]. The methods, limitations, and challenges of relevant literature are summarized in Table I.

### III. PRELIMINARIES

#### A. Network Model

The hospital has implemented medical robots to enhance the quality and efficiency of healthcare delivery within its premises. These robots are deployed to assist the medical staff and provide medical care to inpatients. The system design incorporates five key entities: 1) doctor; 2) medical robot; 3) cloud server; 4) patient; and 5) hospital. Each entity is described as follows.

- 1) *Doctor*: The doctor, possessing the necessary legal qualifications, is responsible for analyzing patient data, making diagnoses, and treating hospitalized patients. Upon successful login into the system, the doctor

engages in communication with the medical robot to convey treatment plans and relevant patient information. This communication serves as a guide to the medical robot for executing specific medical tasks.

- 2) *Medical Robot*: As a crucial aid to healthcare professionals, the medical robot communicates with doctors, the cloud server, and patients. Initially, the medical robot receives data from the doctor, performs preliminary processing, and stores the data as partial blocks in the cloud server. Moreover, the medical robot undergoes an authentication process with the patient. Once the authentication is completed, the robot delivers appropriate medical services to the patient.
- 3) *Cloud Server*: Deployed within the cloud computing environment, the cloud server acts as a device for data storage and processing collected by the medical robot. Furthermore, it functions as a network node of the blockchain, responsible for verifying the partial data blocks uploaded by the medical robots. After successful verification, the cloud server generates complete blocks. The decision to add these blocks to the blockchain is determined through a consensus mechanism implemented by the cloud servers, which evaluates whether the data should be uploaded.
- 4) *Patient*: Before receiving treatment, the patient interacts with the medical robot. Both parties undergo an authentication process to validate their respective identities. Only when both parties are successfully authenticated will the medical robot provide medical care to the patient, enabling the patient to receive the necessary healthcare services.



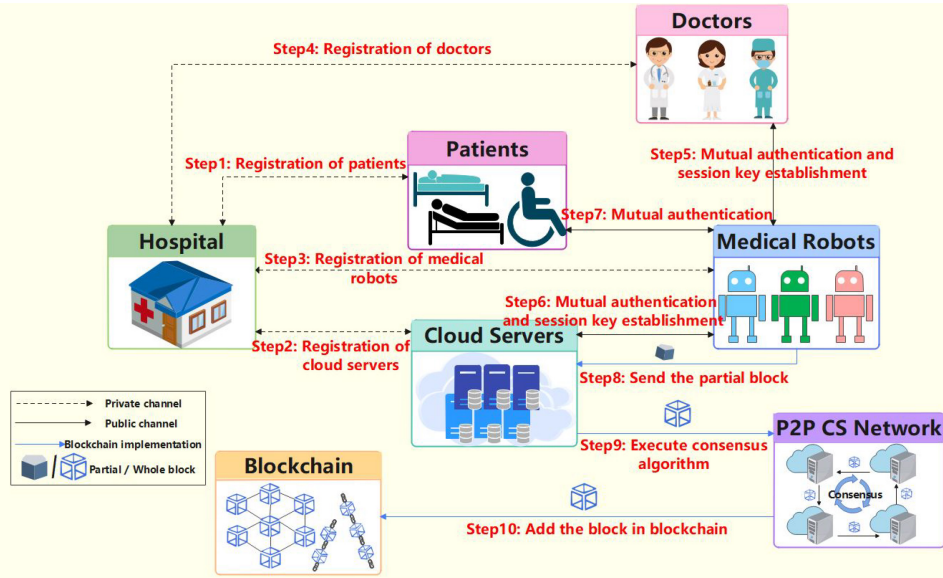


Fig. 2. Overall process flow of the proposed protocol.

- 5) *Hospital*: As the provider of healthcare services, the hospital manages the entities and acts as a trusted hub. The hospital provides registration services for each entity, ensuring that the entity has a legitimate identity in the system. Only after successful registration doctors, medical robots, cloud servers, and patients can participate in the communication process and interact and collaborate accordingly.

#### B. Adversary Model

In the context of the protocol's communication occurring in the public channel, it is crucial to consider potential vulnerabilities and threats posed by adversaries. In this section, we thoroughly examine the "Dolev-Yao" (DY) threat model [45] and the "Canetti-Krawczyk" (CK) adversary model [46] to define the capabilities of adversary  $\mathcal{A}$ . The defined capabilities that adversary  $\mathcal{A}$  can possess are as follows.

- 1) *Eavesdropping, Interception, Tampering, and Deletion*: Adversary  $\mathcal{A}$  is capable of eavesdropping on the communication, intercepting the transmitted information, tampering with the data, and deleting the information exchanged within the public channel.
- 2) *Smart Device Data Acquisition*: Adversary  $\mathcal{A}$  has the ability to obtain data present in the user's smart device by leveraging power analysis techniques.
- 3) *Brute Force Cracking*: Adversary  $\mathcal{A}$  can use brute force cracking to guess a doctor's password or identity. However, it is essential to note that adversary  $\mathcal{A}$  cannot attempt both simultaneously but is limited to one of these options.
- 4) *Insider Access to Cloud Server*: Adversary  $\mathcal{A}$  can act as an insider with access to the cloud server, thereby obtaining internal data. However, to highlight that adversary  $\mathcal{A}$  must not be able to launch an attack on the trusted institution, i.e., the hospital, to acquire the stored data.

The objective of establishing this adversary model is to delineate the capabilities assigned to potential attackers.

Subsequently, this model will provide the groundwork for our ensuing security analyses. The formal security analysis in Section V-A and the informal security analysis in Section V-B are methodically correlated with this adversary model.

#### IV. PROPOSED PROTOCOL

This section provides a detailed exposition of the proposed protocol, encompassing five participating entities: 1) the hospital; 2) doctors; 3) medical robots; 4) cloud servers; and 5) patients. The workflow diagram of the protocol is depicted in Fig. 2. The protocol unfolds through three distinct phases: 1) the registration phase (steps 1–4); 2) the authentication and key agreement phase (steps 5–7); and 3) the blockchain implementation phase (steps 8–10). During the registration phase, participants, including patients (step 1), cloud servers (step 2), medical robots (step 3), and doctors (step 4), register with the hospital sequentially through a secure channel. Only after successful registration as legitimate entities can they engage in the communication process of the protocol. The authentication and key agreement phase involves the doctor and the medical robot authenticating each other and establishing a shared SK (step 5). Subsequently, the medical robot and the cloud server authenticate with each other, negotiating another shared SK (step 6). Additionally, the medical robot and the patient authenticate to confirm each other's identities (step 7). In the blockchain implementation phase, the medical robot transmits partial blocks to the cloud server, which refines these partial blocks into complete blocks (step 8). Multiple cloud servers execute consensus algorithms among themselves (step 9) to ultimately append the blocks to the blockchain (step 10). The notations employed in the protocol are detailed in Table II.

##### A. Registration Phase

The registration phase is responsible for granting legal authorization to entities participating in the protocol, including patients, cloud servers, medical robots, and doctors.

TABLE II  
NOTATIONS USED IN THE PROPOSED PROTOCOL

| Notations                      | Descriptions                               |
|--------------------------------|--------------------------------------------|
| $HP$                           | Hospital                                   |
| $DR_i$                         | $i^{th}$ doctor                            |
| $MR_j$                         | $j^{th}$ medical robot                     |
| $CS_k$                         | $k^{th}$ cloud server                      |
| $PT_c$                         | $c^{th}$ patient                           |
| $Bio_i$                        | $DR_i$ 's biometric information            |
| $ID_i, PW_i$                   | $DR_i$ 's identity, password               |
| $RID_j, SID_k, TID_c$          | Identities of $MR_j, CS_k$ and $PT_c$      |
| $K_H$                          | Secret key of $HP$                         |
| $k_{MR_j}, k_{CS_k}, k_{PT_c}$ | Private keys of $MR_j, CS_k$ and $PT_c$    |
| $G$                            | A base point in an elliptic curve          |
| $PK_Y$                         | Public key of $Y$ ; $PK_Y = k_Y \cdot G$   |
| $T_r$                          | Timestamps, where $r = 1, 2, 3, 4, 5, 6$   |
| $n_s, r_s, a_s$                | Random numbers, where $s = 1, 2, 3, 4$     |
| $SK$                           | Session key                                |
| $\mathcal{A}$                  | The adversary                              |
| $Gen(\cdot)$                   | Generate function of fuzzy extractor       |
| $Rep(\cdot)$                   | Reproduce function of fuzzy extractor      |
| $Enc_k(\cdot)/Dec_k(\cdot)$    | Symmetric encryption/decryption with $k$   |
| $ECDSA$                        | Elliptic Curve Digital Signature Algorithm |
| $h(\cdot)$                     | One-way hash operation                     |
| $\parallel$                    | Concatenation operation                    |
| $\oplus$                       | XOR operation                              |
| $\Rightarrow$                  | The private channel                        |
| $\rightarrow$                  | The public channel                         |

1) *Patient Registration Phase*: Assume that a patient  $PT_c$  desires to register with  $HP$ .  $PT_c$  initiates the registration process by sending a registration request to  $HP$ . Upon receiving the request,  $HP$  verifies and authorizes  $PT_c$  to register as a legitimate user eligible to participate in the protocol. The step-by-step description of the patient registration phase is as follows.

- 1)  $PT_c$  enters identity  $TID_c$  and utilizes a randomly generated number  $a_1$  to compute the pseudo-identity  $PID_c = h(TID_c \parallel a_1)$ . Subsequently,  $PT_c$  securely transmits  $\{PID_c, a_1\}$  to  $HP$  through a private channel.
- 2)  $HP$  validates whether  $PT_c$  is already registered by checking  $PID_c$ . If  $PT_c$  is not registered,  $HP$  generates a random number  $a_2$  and computes  $A_i = h(PID_c \parallel K_H \parallel a_2)$  and  $B_i = h(A_i \parallel a_1) \oplus PID_c$ .  $HP$  then stores  $\{PID_c, a_1, a_2\}$  in the hospital database and sends  $A_i$  to  $PT_c$ .
- 3)  $PT_c$  receives  $A_i$  from  $HP$ .  $PT_c$  computes  $H_i = A_i \oplus h(PID_c \parallel k_{PT_c})$  and  $W_i = a_1 \oplus h(H_i \parallel A_i)$ .  $PT_c$  stores  $\{PID_c, H_i, W_i\}$  in his smart device for future use.

2) *Cloud Server Registration Phase*: Here, we describe the process of registering a cloud server, denoted as  $CS_k$ , with  $HP$ . Upon approval of the registration request by  $HP$ ,  $CS_k$  becomes a legitimate participant in the protocol. The following steps provide a detailed explanation of this phase.

- 1)  $CS_k$  initiates the registration process by providing its own identity  $SID_k$  and generating a random number  $a_3$ . Subsequently,  $CS_k$  computes  $CID_k = h(SID_k \parallel a_3)$  and sends the resulting  $CID_k$  to  $HP$ .
- 2)  $HP$  verifies the registration status of  $CS_k$  by checking the value of  $CID_k$ . Furthermore,  $HP$  generates a random number  $a_4$  and utilizes it to compute  $HC_i = h(CID_k \parallel a_4)$  and  $C_i = h(HC_i \parallel K_H \parallel a_4)$ .  $HP$  stores  $\{CID_k, a_4\}$  in the database and transmits  $\{HC_i, C_i, a_4, PID_c\}$  to  $CS_k$ .

- 3) Upon receiving the transmitted data,  $CS_k$  performs the following computations sequentially:  $S_i = C_i \oplus h(CID_k \parallel k_{CS_k})$  and  $Q_i = a_4 \oplus h(S_i \parallel C_i \parallel k_{CS_k})$ . Finally,  $CS_k$  stores  $\{PID_c, CID_k, S_i, Q_i\}$  in its own database.

3) *Medical Robot Registration Phase*: The registration of the medical robot  $MR_j$  with the hospital entity  $HP$  is an essential step that must be completed for  $MR_j$  to become a recognized participant in the protocol. The following detailed process outlines the registration phase of  $MR_j$ .

- 1)  $MR_j$  provides its identity  $RID_j$  and computes  $MID_j = h(RID_j \parallel r_1)$  using a randomly generated number  $r_1$ . Subsequently,  $MR_j$  securely sends  $MID_j$  to  $HP$  through a private channel.
- 2)  $HP$  receives  $MID_j$  and first checks if it already exists in the database. If  $MID_j$  is not found in the database,  $HP$  proceeds with the registration process.  $HP$  generates a random number  $r_2$  and computes  $A_i = h(PID_c \parallel K_H \parallel a_2)$ ,  $M_i = h(MID_j \parallel r_2)$ , and  $L_i = h(M_i \parallel r_2 \parallel PID_c) \oplus A_i$ . Additionally, in this phase,  $HP$  computes  $HC_i = h(CID_k \parallel a_4)$ ,  $C_i = h(HC_i \parallel K_H \parallel a_4)$ , and  $P_i = h(CID_k \parallel r_2) \oplus a_4$ .  $HP$  stores  $\{M_i, MID_j\}$  in the database. To proceed with  $MR_j$  registration,  $HP$  sends  $\{r_2, a_1, a_2, L_i, CID_k, C_i, PID_c, P_i\}$  to  $MR_j$ .
- 3)  $MR_j$  computes  $K_i = r_2 \oplus h(MID_j \parallel k_{MR_j})$ ,  $O_i = Enc_{(K_i \parallel k_{MR_j})}(C_i \parallel P_i)$ ,  $R_i = h(K_i \parallel a_2) \oplus a_1$ , and  $V_i = h(PID_c \parallel r_2) \oplus a_2$  based on the parameters received from  $HP$ .  $MR_j$  stores  $\{PID_c, CID_k, MID_j, K_i, O_i, V_i, R_i, L_i\}$  in its database for future use.

4) *Doctor Registration Phase*: Let us consider the scenario where a doctor, denoted as  $DR_i$ , intends to register with  $HP$ . Once  $HP$  receives the registration request from  $DR_i$ , it authorizes  $DR_i$  if the request is valid and the doctor is certified. Only certified doctors are allowed to participate in the protocol. The subsequent section provides a detailed description of the registration process for  $DR_i$ .

- 1) When  $DR_i$  initiates the registration process, they provide their identity  $ID_i$ , password  $PW_i$ , and biometric information  $Bio_i$ . Subsequently,  $DR_i$  computes  $Gen(Bio_i) = (\delta_i, \tau_i)$  using the fuzzy extractor function. Additionally,  $DR_i$  computes  $DC_i = h(ID_i \parallel \delta_i)$ . Furthermore,  $DR_i$  computes  $DP_i = h(ID_i \parallel r_3) \oplus h(PW_i \parallel \delta_i)$  and  $DH_i = h(DP_i \parallel \delta_i \parallel ID_i)$  based on the random number  $r_3$  and the extracted random key  $\delta_i$ . Finally,  $DR_i$  sends  $\{ID_i, DP_i, DC_i\}$  to  $HP$ .
- 2)  $HP$  receives the data and first checks whether  $DC_i$  has been stored in the database earlier. If the doctor has not been registered, then  $HP$  generates a random number  $r_4$ . Next,  $HP$  computes  $D_i = h(ID_i \parallel r_4)$ ,  $DM_i = h(D_i \parallel MID_j \parallel M_i)$ ,  $E_i = Enc_{(DP_i)}(DM_i)$ .  $HP$  stores  $DC_i$  in the database and transmits  $\{E_i, r_4, PID_c, MID_j\}$  to  $DR_i$ .
- 3) After receiving the message from  $HP$ ,  $DR_i$  stores  $\{MID_j, PID_c, E_i, DH_i, \tau_i, r_3, r_4\}$  in  $DR_i$ 's smart device.

## B. Authentication and Key Agreement Phase

The authentication and key agreement phase is essential for establishing trust and ensuring secure communication between entities. This phase involves three processes: 1) the

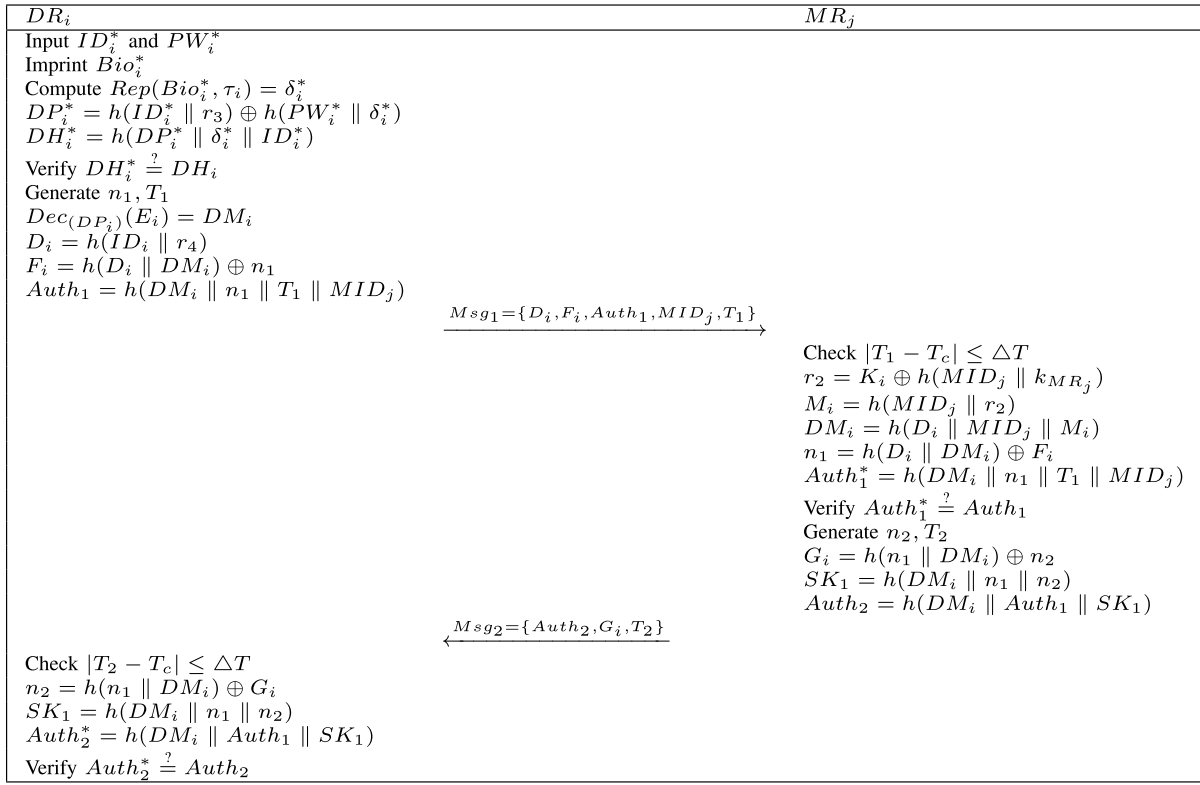


Fig. 3. Authentication and SK establishment between  $DR_i$  and  $MR_j$ .

authentication and key agreement phase between a doctor ( $DR_i$ ) and a medical robot ( $MR_j$ ); 2) the authentication and key agreement phase between  $MR_j$  and a cloud server ( $CS_k$ ); and 3) the mutual authentication phase between  $MR_j$  and a patient ( $PT_c$ ).

1) *Authentication and Key Agreement Phase Between Doctor and Medical Robot:* In this phase,  $DR_i$  authenticates with  $MR_j$  to establish a SK and facilitate secure information transmission. This allows the doctor to utilize the medical robot to effectively treat the designated patient. The following steps describe the authentication and key agreement phase between  $DR_i$  and  $MR_j$ , as depicted in Fig. 3.

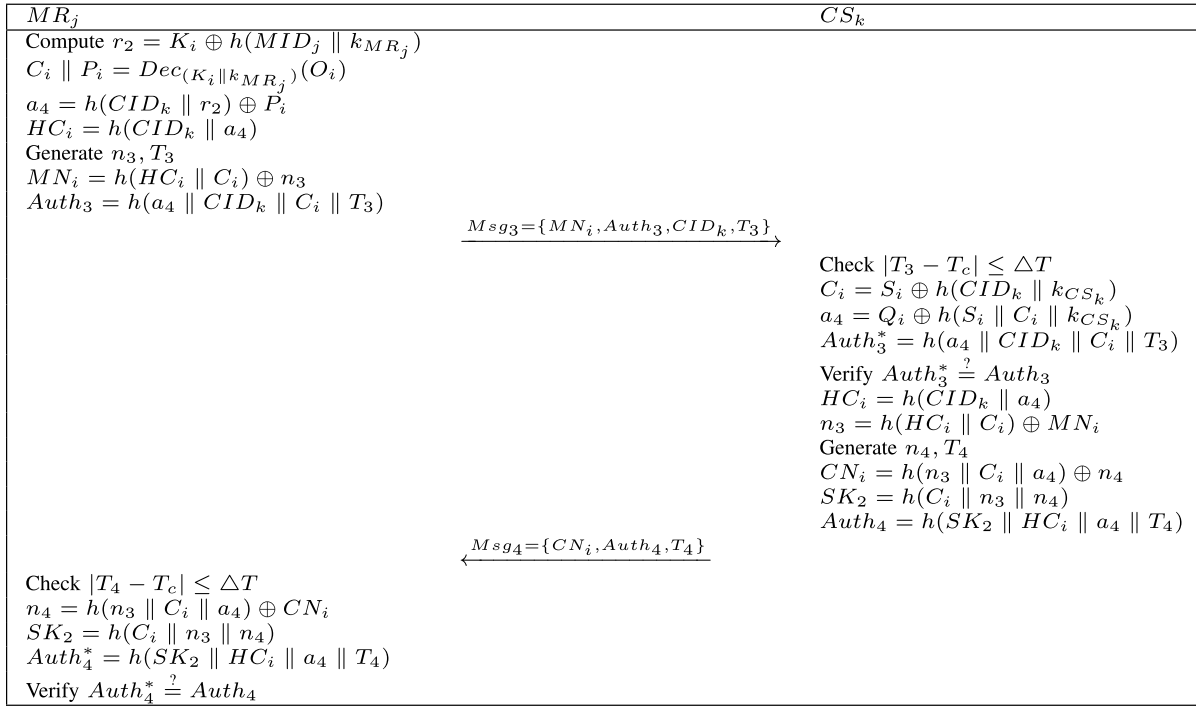
- 1)  $DR_i$  provides their identity  $ID_i^*$ , password  $PW_i^*$ , and biometric information  $Bio_i^*$ . Using the fuzzy extractor function,  $DR_i$  computes the reproduction function  $Rep(Bio_i^*, \tau_i) = \delta_i^*$ . Utilizing the random number  $r_3$  stored in their smart device,  $DR_i$  computes  $DP_i^* = h(ID_i^* || r_3) \oplus h(PW_i^* || \delta_i^*)$  and  $DH_i^* = h(DP_i^* || \delta_i^* || ID_i^*)$ . By comparing  $DH_i^*$  with  $DH_i$ , it can be determined whether the authentication is successful. If they match, it indicates a successful login by  $DR_i$ . Subsequently,  $DR_i$  generates the current timestamp  $T_1$  and a random number  $n_1$ . Decrypting with  $DP_i$ ,  $DR_i$  obtains  $DM_i$ . Then,  $DR_i$  computes  $D_i = h(ID_i || r_4)$ ,  $F_i = h(D_i || DM_i) \oplus n_1$ , and  $Auth_1 = h(DM_i || n_1 || T_1 || MID_j)$ .  $DR_i$  sends the message  $Msg_1 = \{D_i, F_i, Auth_1, MID_j, T_1\}$  to  $MR_j$  via the public channel.
- 2) Upon receiving the data from  $DR_i$ ,  $MR_j$  verifies the freshness of the timestamp  $T_1$ . Subsequently,  $MR_j$  computes  $r_2 = K_i \oplus h(MID_j || k_{MR_j})$  and  $M_i = h(MID_j || r_2)$ .

Next,  $MR_j$  computes  $DM_i = h(D_i || MID_j || M_i)$ ,  $n_1 = h(D_i || DM_i) \oplus F_i$ , and  $Auth_1^* = h(DM_i || n_1 || T_1 || MID_j)$ . Furthermore,  $MR_j$  verifies the identity of  $DR_i$  by comparing the values of  $Auth_1$  and  $Auth_1^*$ . After successful verification,  $MR_j$  generates a random number  $n_2$  and a timestamp  $T_2$ .  $MR_j$  computes  $G_i = h(n_1 || DM_i) \oplus n_2$ , the session key  $SK_1 = h(DM_i || n_1 || n_2)$ , and the parameter  $Auth_2 = h(DM_i || Auth_1 || SK_1)$ .  $MR_j$  sends the message  $Msg_2 = \{Auth_2, G_i, T_2\}$  to  $DR_i$ .

- 3)  $DR_i$  first checks if the timestamp  $T_2$  satisfies  $|T_2 - T_c| \leq \Delta T$ . If the condition is met,  $DR_i$  computes  $n_2 = h(n_1 || DM_i) \oplus G_i$ . Finally,  $DR_i$  computes the session key  $SK_1 = h(DM_i || n_1 || n_2)$  and  $Auth_2^* = h(DM_i || Auth_1 || SK_1)$ . By verifying whether  $Auth_2^* \stackrel{?}{=} Auth_2$ ,  $DR_i$  can confirm the identity of  $MR_j$  and the successful establishment of the correct SK.

2) *Authentication and Key Agreement Phase Between Medical Robot and Cloud Server:* Here, we will provide a detailed description of the authentication and key agreement phase between a Medical Robot ( $MR_j$ ) and a Cloud Server ( $CS_k$ ). This phase aims to establish a secure communication channel by conducting authentication and negotiating a unified SK. Fig. 4 illustrates the authentication process and key agreement between  $MR_j$  and  $CS_k$ .

- 1) First,  $MR_j$  performs the computation  $r_2 = K_i \oplus h(MID_j || k_{MR_j})$  and obtains  $(C_i || P_i)$  by decrypting  $O_i$ . Subsequently,  $MR_j$  calculates  $a_4 = h(CID_k || r_2) \oplus P_i$  and  $HC_i = h(CID_k || a_4)$ . Furthermore,  $MR_j$  generates a random number  $n_3$  and obtains the current timestamp  $T_3$ . Then,  $MR_j$  computes  $MN_i = h(HC_i || C_i) \oplus n_3$  and

Fig. 4. Authentication and SK establishment between  $MR_j$  and  $CS_k$ .

$Auth_3 = h(a_4 \| CID_k \| C_i \| T_3)$ . Finally,  $MR_j$  sends the message  $Msg_3 = \{MN_i, Auth_3, CID_k, T_3\}$  to  $CS_k$ .

- 2) Upon receiving the data from  $MR_j$ ,  $CS_k$  initiates by checking the freshness of the timestamp  $T_3$ . Then,  $CS_k$  computes  $C_i = S_i \oplus h(CID_k \| k_{CS_k})$ ,  $a_4 = Q_i \oplus h(S_i \| C_i \| k_{CS_k})$ , and  $Auth_3^* = h(a_4 \| CID_k \| C_i \| T_3)$  in sequence. Subsequently,  $CS_k$  verifies if  $Auth_3^* \stackrel{?}{=} Auth_3$ . If they are equal, it demonstrates that  $MR_j$  has successfully passed the authentication process of  $CS_k$ . Next,  $CS_k$  computes  $HC_i = h(CID_k \| a_4)$ ,  $n_3 = h(HC_i \| C_i) \oplus MN_i$ , generates a random number  $n_4$ , and obtains the current timestamp  $T_4$ . Following that,  $CS_k$  calculates  $CN_i = h(n_3 \| C_i \| a_4) \oplus n_4$ , derives the session key  $SK_2 = h(C_i \| n_3 \| n_4)$ , computes  $Auth_4 = h(SK_2 \| HC_i \| a_4 \| T_4)$ , and sends the message  $Msg_4 = \{CN_i, Auth_4, T_4\}$  to  $MR_j$ .
- 3) After receiving  $Msg_4$ ,  $MR_j$  checks the timestamp by evaluating  $|T_4 - T_c| \leq \Delta T$ . Subsequently,  $MR_j$  performs the computations  $n_4 = h(n_3 \| C_i \| a_4) \oplus CN_i$ ,  $SK_2 = h(C_i \| n_3 \| n_4)$ , and  $Auth_4^* = h(SK_2 \| HC_i \| a_4 \| T_4)$ . Finally,  $MR_j$  verifies whether  $Auth_4^* \stackrel{?}{=} Auth_4$ . If they are equal, it indicates that  $CS_k$  has successfully passed the authentication process of  $MR_j$ , and both parties have established the same SK.

3) *Mutual Authentication Phase Between Medical Robot and Patient*: Prior to providing medical services to  $PT_c$ , it is essential for  $MR_j$  to ensure the accuracy of the services. Therefore,  $MR_j$  initiates a process of mutual identity verification with  $PT_c$  before implementing any treatment measures. Only upon confirming the correct identity of the other party will  $MR_j$  proceed with delivering the relevant medical services to  $PT_c$ . The detailed procedure for mutual authentication between  $MR_j$  and  $PT_c$  is outlined below. The specific steps

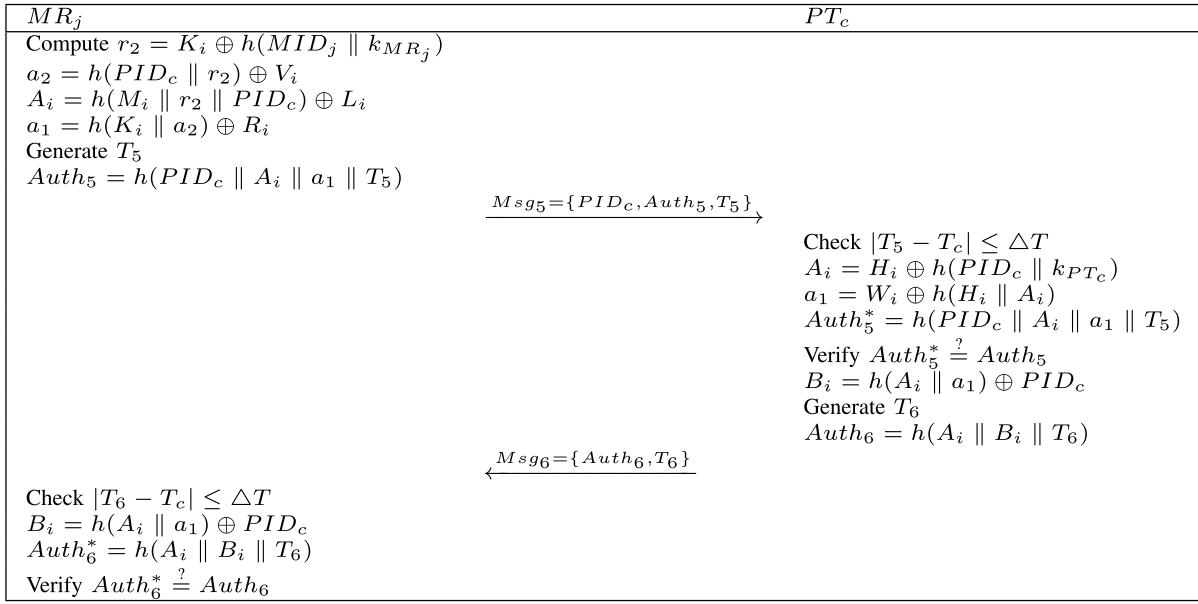
involved in the mutual authentication phase are depicted in Fig. 5.

- 1) Initially,  $MR_j$  calculates  $r_2 = K_i \oplus h(MID_j \| k_{MR_j})$ , and subsequently determines  $a_2 = h(PID_c \| r_2) \oplus V_i$ . Following this,  $MR_j$  computes  $A_i = h(M_i \| r_2 \| PID_c) \oplus L_i$ , and  $a_1 = h(K_i \| a_2) \oplus R_i$ . Additionally,  $MR_j$  generates the current timestamp  $T_5$ . Further,  $MR_j$  computes  $Auth_5 = h(PID_c \| A_i \| a_1 \| T_5)$  and transmits the message  $Msg_5 = \{PID_c, Auth_5, T_5\}$  to  $PT_c$ .
- 2) Upon receiving  $Msg_5$ ,  $PT_c$  verifies the condition  $|T_5 - T_c| \leq \Delta T$ . If the condition is satisfied,  $PT_c$  proceeds to compute  $A_i = H_i \oplus h(PID_c \| k_{PT_c})$ ,  $a_1 = W_i \oplus h(H_i \| A_i)$ , and  $Auth_5^* = h(PID_c \| A_i \| a_1 \| T_5)$ . Subsequently,  $PT_c$  checks whether  $Auth_5^*$  is equal to  $Auth_5$ . If the equality holds, it indicates that  $MR_j$  has successfully passed the authentication process of  $PT_c$ .  $PT_c$  further computes  $B_i = h(A_i \| a_1) \oplus PID_c$ , generates the current timestamp  $T_6$ , and computes  $Auth_6 = h(A_i \| B_i \| T_6)$ . Finally,  $PT_c$  sends the message  $Msg_6 = \{Auth_6, T_6\}$  to  $MR_j$ .
- 3) Upon receiving  $Msg_6$  from  $PT_c$ ,  $MR_j$  first verifies the freshness of the timestamp  $T_6$ . Subsequently,  $MR_j$  calculates  $B_i = h(A_i \| a_1) \oplus PID_c$ ,  $Auth_6^* = h(A_i \| B_i \| T_6)$ , and verifies whether  $Auth_6^* \stackrel{?}{=} Auth_6$ . If the equation holds, it confirms that  $PT_c$  has successfully passed the authentication process of  $MR_j$ . This indicates that both  $MR_j$  and  $PT_c$  have mutually verified each other's identities successfully.

### C. Blockchain Implementation Phase

This section details constructing blocks, verifying blocks, and adding blocks to the blockchain.



Fig. 5. Mutual authentication between  $MR_j$  and  $PT_c$ .

1) *Constructing the Block*: Upon receiving data from  $DR_i$ ,  $MR_j$  processes the data and generates partial blocks that are then uploaded to  $CS_k$ . The role of  $CS_k$  is to construct these partial blocks into complete blocks. The process of constructing blocks is described in detail below.

- 1) During the communication phase,  $DR_i$  and  $MR_j$  engage in mutual authentication and establish a shared session key  $SK_1$ . When  $DR_i$  sends data to  $MR_j$ , it encrypts the data using the shared session key  $SK_1$ . Upon receiving the encrypted data,  $MR_j$  decrypts it using  $SK_1$ . Once the data is successfully decrypted,  $MR_j$  generates  $nt$  transactions, denoted as  $Tx_1, Tx_2, \dots, Tx_{nt}$ . Subsequently,  $MR_j$  encrypts these transactions using its own public key  $PK_{MR_j}$ , resulting in encrypted transactions  $E_{PK_{MR_j}}(Tx_1), E_{PK_{MR_j}}(Tx_2), \dots, E_{PK_{MR_j}}(Tx_{nt})$ .
- 2) Next,  $MR_j$  constructs a partial block  $ParBlock_l$  based on these transactions.  $ParBlock_l$  consists of  $n$  encrypted transactions  $E_{PK_{MR_j}}(Tx_1), E_{PK_{MR_j}}(Tx_2), \dots, E_{PK_{MR_j}}(Tx_{nt})$ , the elliptic curve digital signature algorithm (ECDSA) signature  $Sig_{Block_l}$  of the encrypted transactions, the block version  $BV_l$ , the Merkle tree root  $MT_l$  of the encrypted transactions, the timestamp value  $TV_l$  indicating the block creation time, the block owner  $OB_l$ , and the public key  $PK_{MR_j}$  of the block owner for verification. The  $Sig_{Block_l}$  is computed using the private key  $k_{MR_j}$  of  $MR_j$  and the ECDSA for generating a digital signature of the cryptographic transactions.  $MR_j$  securely transmits the created partial block  $ParBlock_l$  to the cloud server  $CS_k$  within the peer-to-peer cloud server (P2P CS) network.
- 3) Upon receiving the partial block  $ParBlock_l$ ,  $CS_k$  verifies the integrity of  $ParBlock_l$  by checking if the public key  $PK_{MR_j}$  contained in  $ParBlock_l$  can successfully verify the signature  $Sig_{Block_l}$ . Once the verification is successful,  $CS_k$  adds the previous block hash  $PB_l$  to the partial block and calculates the current block

| Block Header                          |                                                  |
|---------------------------------------|--------------------------------------------------|
| Block Version                         | $BV_l$                                           |
| Previous Block Hash                   | $PB_l$                                           |
| Merkle Tree Root                      | $MT_l$                                           |
| Timestamp Value                       | $TV_l$                                           |
| Owner of Block                        | $OB_l$                                           |
| Owner's Public Key                    | $PK_{MR_j}$                                      |
| Block Payload(Encrypted Transactions) |                                                  |
| List of Encrypted Transactions        | $\{E_{PK_{MR_j}}(Tx_x   x = 1, 2, \dots, n_t)\}$ |
| Signature on Block using ECDSA        | $Sig_{Block_l}$                                  |
| Current Block Hash                    | $BH_l$                                           |

Fig. 6. Structure of block  $Block_l$ .

information  $\{BV_l, PB_l, MT_l, TV_l, OB_l, PK_{MR_j}, \text{block payload (encrypted transactions)}\}$  to obtain the current block hash  $BH_l$ . Finally,  $CS_k$  completes the partial block  $ParBlock_l$  by transforming it into a complete block  $Block_l$ , as depicted in Fig. 6.

2) *Verify the Block and Add It to the Blockchain*: After the cloud server creates block  $Block_l$ , two tasks must be completed to add the block to the blockchain. First, the leader node  $L$  is selected using the leader election algorithm. The leader node  $L$  plays a crucial role in verifying and adding the block to the blockchain. The existing leader election mechanism proposed in [47] is utilized to perform the election process. Second, the block needs to be verified, and consensus for adding it to the blockchain is achieved through the practical Byzantine fault tolerant (PBFT) consensus algorithm based on voting. Additionally, it is assumed that each cloud server  $CS_k$  ( $k = 1, 2, \dots, n_{cs}, L \neq CS_k$ ) in the P2P CS network possesses an Elliptic curve cryptography (ECC)-based private-public key pair  $(k_{CS_k}, PK_{CS_k})$ . The public key  $PK_{CS_k}$  is publicly available within the network, where  $PK_{CS_k} = k_{CS_k} \cdot G$  and  $n_{cs}$  represents the number of cloud servers in the P2P CS network. Notably, a maximum of 1/3 of the cloud server nodes are allowed to fail or become compromised within this network. This process

**Algorithm 1** Block Validation and Addition in the Blockchain

**Require:** A whole block  $Block_l$  as shown in Figure 6; private-public key pairs  $(k_{CS_k}, PK_{CS_k} = k_{CS_k} \cdot G)$  for all cloud servers  $CS_k$  in the P2P CS network.

**Ensure:** Verification of block  $Block_l$  and its addition to the blockchain.

```

1: First, the leader ( $L$ ) is selected among the peer nodes (cloud servers) in the P2P CS network.  $L$  holds a block  $Block_l$ .
2:  $L$  initializes  $VCount \leftarrow 0$ .
3:  $L$  chooses the random number  $RN_k$  for  $CS_k$  and generates the current timestamp value  $TV_k$ .
4:  $L$  encrypts  $RN_k$ ,  $TV_k$ , and  $VReq_k$  as  $EReq = E_{PK_{CS_k}}(RN_k, TV_k, VReq_k)$ .
5:  $L$  broadcasts the message  $SMsg_k = \{Block_l, EReq, TV_k\}$ .
6: At the time of  $TV_k^*$ ,  $CS_k$  receives  $SMsg_k$  from  $L$ .
7: for  $CS_k$  do
8:   if  $|TV_k^* - TV_k| \leq \Delta T$  then
9:     Calculate the Merkle tree root  $MT_l^*$  based on received  $Block_l$ .
10:    if  $MT_l^* \neq MT_l$  then
11:      Terminate the consensus process.
12:    else
13:      Calculate the block hash  $BH_l^*$  of  $Block_l$ ,  $BH_l^* = h(BV_l \parallel PB_l \parallel MT_l^* \parallel TV_l \parallel OB_l \parallel PK_{MR_j} \parallel E_{PK_{MR_j}}(Tx_1) \parallel E_{PK_{MR_j}}(Tx_2) \parallel \dots \parallel E_{PK_{MR_j}}(Tx_{n_t}))$ .
14:      if  $BH_l^* \neq BH_l$  then
15:        Signature verification on  $Block_l$  using the message  $msg = BH_l^*$ .
16:        if signature verification is valid then
17:          Decrypt  $EReq$  to get  $RN_k^*$ ,  $TV_k^{**}$ , and  $VReq_k^*$ .
18:          if  $TV_k^{**} = TV_k$  then
19:            Send the message  $RMsg_k = E_{PK_L}(RN_k^*, VSt_k, VResp_k)$  to  $L$ .
20:          end if
21:        end if
22:      end if
23:    end if
24:  end if
25: end for
26: for each  $RMsg_k$  received from the responders  $CS_k$  do
27:    $L$  calculates  $(RN_k^{**}, VSt_k^*, VResp_k^*) = D_{k_L}[RMsg_k]$ .
28:   if  $((RN_k^{**} = RN_k)$  and  $(VSt_k^* = valid)$  and  $(VResp_k^* = valid))$  then
29:      $L$  sets  $VCount = VCount + 1$ .
30:   end if
31: end for
32: if  $(VCount > 2 * (n_{cs} - 1)/3 + 1)$  then
33:   Add block  $Block_l$  to the blockchain.
34:   Broadcast the block's commit status to all followers in the P2P CS network.
35: end if

```

phase is illustrated in Algorithm 1. A detailed description of Algorithm 1 follows.

- 1) Initially, one cloud server is selected as the leader node  $L$  from all available cloud servers in the P2P CS network. The block  $Block_l$  is owned by  $L$ .  $L$  sets the initial value of the vote count  $VCount$  of  $CS_k$  to 0. For each  $CS_k$  in the P2P CS network,  $L$  chooses a random number  $RN_k$  and generates the current timestamp value  $TV_k$ .  $L$  encrypts  $RN_k$ ,  $TV_k$ , and the voting request  $VReq_k$  using the public key  $PK_{CS_k}$  of  $CS_k$ . The encryption results in  $EReq = E_{PK_{CS_k}}(RN_k, TV_k, VReq_k)$ .  $L$  broadcasts the message  $SMsg_k = \{Block_l, EReq, TV_k\}$  to  $CS_k$ . Assume that  $CS_k$  receives the message  $SMsg_k$  from  $L$  at time  $TV_k^*$ .
- 2) Upon receiving the message  $SMsg_k$  from  $L$ ,  $CS_k$  first checks the freshness of  $TV_k^*$ . If  $TV_k^*$  is fresh,  $CS_k$

calculates the Merkle tree root  $MT_l^*$  based on the transactions stored in the block payload of the received  $Block_l$ . Subsequently,  $CS_k$  verifies whether  $MT_l^*$  matches the expected  $MT_l$ . If they do not match,  $CS_k$  terminates the consensus process. Otherwise,  $CS_k$  computes the block hash  $BH_l^* = h(BV_l \parallel PB_l \parallel MT_l^* \parallel TV_l \parallel OB_l \parallel PK_{MR_j} \parallel E_{PK_{MR_j}}(Tx_1) \parallel E_{PK_{MR_j}}(Tx_2) \parallel \dots \parallel E_{PK_{MR_j}}(Tx_{n_t}))$  of  $Block_l$ .  $CS_k$  verifies whether  $BH_l^*$  matches the expected  $BH_l$ . If they match,  $CS_k$  utilizes the ECDSA signature mechanism and the public key  $PK_{MR_j}$  of the medical robot in  $Block_l$  to verify the signature of the message  $msg = BH_l^*$ . If the signature verification is successful,  $CS_k$  decrypts the encrypted  $EReq = E_{PK_{CS_k}}(RN_k, TV_k, VReq_k)$  using its private key  $k_{CS_k}$ . Suppose the resulting  $TV_k^{**}$  obtained by  $CS_k$  decryption

is equal to the received  $TV_k$ . In that case,  $CS_k$  sends the message  $RMsg_k = E_{PK_L}(RN_k^*, VSt_k, VResp_k)$  to  $L$ , where  $VSt_k$  and  $VResp_k$  represent the verification status of  $Block_l$  and the voting reply corresponding to  $VReq_k^*$ , respectively.

- 3) By decrypting  $RMsg_k$ ,  $L$  obtains  $(RN_k^{**}, VSt_k^*, \text{ and } VResp_k^*)$ . If  $RN_k^{**} = RN_k$ , and both  $VSt_k^*$  and  $VResp_k^*$  are valid, then  $L$  increments the value of the vote count  $VCount$  by 1.
- 4) If the block verification status is valid and more than 2/3 of the cloud server, then nodes respond with a positive vote, which means that the block is successfully added to the blockchain. Additionally, other cloud servers in the peer nodes will add the block to their distributed ledgers.

## V. SECURITY ANALYSIS

This section conducts a comprehensive security analysis of the protocol, incorporating a formal security method. Subsequently, we evaluate the protocol's resilience against various malicious attacks and assess its capacity to meet diverse security requirements.

### A. Formal Security Analysis

The ROR model is a widely used approach in the formal analysis of authentication protocols, which is designed to evaluate the security of protocols under actual attacks and random situations [48]. The real situation considers all the attacks the protocol may face, modeling the challenges in a real environment. In contrast, protocol execution in the random case is generated by a stochastic process without the involvement of an actual attacker. We can identify potential vulnerabilities and attack paths by comparing the protocol's performance in these two cases. The reasons for using the ROR model in mutual authentication and key agreement protocols include its comprehensiveness, standardization, and theoretical guarantees. This approach provides a consistent framework for analyzing the security of protocols, helps identify weaknesses in protocol design, and ensures that the protocols are theoretically strong enough to cope with a wide range of potential security threats.

Here, we employ the ROR model [49], [50], [51] to conduct a formal security analysis of the protocol. The ROR model utilizes an execution mechanism that quantifies the probability of compromising the SK over multiple game rounds. We have defined the symbols used in the proof and game process, including symbols, such as  $\Pi_{DR_i}^x$ ,  $\Pi_{MR_j}^y$ ,  $\Pi_{CS_k}^z$ , etc., as illustrated in Table III. There are five queries that  $\mathcal{A}$  can perform, namely, the *Send* query, *Execute* query, *Hash* query, *Corrupt* query, and *Test* query. The game process comprises five rounds. The first round initiates the game process, followed by the second round, during which  $\mathcal{A}$  eavesdrops on messages transmitted on public channels. The third round involves  $\mathcal{A}$  verifying whether there are hash conflicts. In the fourth round,  $\mathcal{A}$  attempts to steal data stored on smart devices or engages in offline password-guessing operations. The fifth and final round allows  $\mathcal{A}$  to try to crack encryption parameters to obtain the

TABLE III  
PARAMETERS INVOLVED IN THE ROR MODEL

| Parameters     | Descriptions                                                                               |
|----------------|--------------------------------------------------------------------------------------------|
| $\Pi_{DR_i}^x$ | $x$ -th instance of $DR_i$                                                                 |
| $\Pi_{MR_j}^y$ | $y$ -th instance of $MR_j$                                                                 |
| $c$            | $z$ -th instance of $CS_k$                                                                 |
| $\mathcal{A}$  | The adversary                                                                              |
| $Q$            | All participating entities, including $\Pi_{DR_i}^x$ , $\Pi_{MR_j}^y$ , and $\Pi_{CS_k}^z$ |
| $s$            | The String                                                                                 |
| $m$            | Communication messages                                                                     |
| $SK$           | Session key                                                                                |
| $c$            | The result of determining whether $SK$ is in a safe state                                  |
| $sid$          | Session identifier                                                                         |
| $P$            | Our protocol                                                                               |
| $GM_i$         | he $i$ -th round of the game, where $i=0,1,2,3,4$                                          |
| $q_h$          | The number of hash queries executed                                                        |
| $q_s$          | The number of send queries executed                                                        |
| $C, s'$        | Constant parameters in Zipf's law                                                          |
| $l$            | The random biometric key's bit length                                                      |
| $\Omega$       | The IND-CPA symmetric cipher                                                               |

SK. By calculating these probabilities, we demonstrate that our design achieves a high level of security.

1) *Security Model*: The formal security analysis involves three entities: 1)  $DR_i$ ; 2)  $MR_j$ ; and 3)  $CS_k$ . We denote these participants  $\Pi_{DR_i}^x$ ,  $\Pi_{MR_j}^y$ , and  $\Pi_{CS_k}^z$ , respectively. To assess the protocol's security, we consider the query capabilities of the adversary  $\mathcal{A}$ , which are defined as  $Q = \{\Pi_{DR_i}^x, \Pi_{MR_j}^y, \Pi_{CS_k}^z\}$ . The following query operations can be performed by  $\mathcal{A}$  to compromise the protocol's security.

- 1) *Execute* ( $Q$ ):  $\mathcal{A}$  can intercept the communication messages transmitted by  $DR_i$ ,  $MR_j$ , and  $CS_k$  over the public channel. This query represents a passive attack.
- 2) *Hash* ( $s$ ): This query enables  $\mathcal{A}$  to input a string  $s$  and obtain its corresponding hash value using the *Hash* function.
- 3) *Send* ( $Q, m$ ):  $\mathcal{A}$  can send  $m$  to  $Q$  and receive the response message from  $Q$ .
- 4) *Corrupt* ( $Q$ ):  $\mathcal{A}$  can gain access to the secret parameters of a specific entity, such as the long-term key, temporarily generated information, and data stored in the smart device.
- 5) *Test* ( $Q$ ): The *Test* query enables the assessment of the semantic security of  $SK$  established during the communication between entities.  $\mathcal{A}$  performs this query by flipping a coin  $c$ , where  $c$  can only yield two results: a) 0 or b) 1. If  $c = 1$ ,  $\mathcal{A}$  can retrieve the correct SK, while if  $c = 0$ ,  $\mathcal{A}$  can only obtain a random string of the same length as  $SK$ .

2) *Semantic Security*: Here, we present the relevant definition of semantic security for the proposed protocol.

*Accepted State*: The session identifier  $sid$  is generated when the exchanged messages are delivered in a specific order. Upon receiving the last expected message of the protocol, entity  $Q$  transitions into the accept state.

*Partners*: To establish the partnership between two entities, such as  $\Pi_{DR_i}^x$  and  $\Pi_{MR_j}^y$ , the following conditions must be satisfied: 1) both  $\Pi_{DR_i}^x$  and  $\Pi_{MR_j}^y$  are in an acceptable state; 2)  $\Pi_{DR_i}^x$  and  $\Pi_{MR_j}^y$  successfully authenticate each other in a session with the same  $sid$ ; and 3) both  $\Pi_{DR_i}^x$  and  $\Pi_{MR_j}^y$  are companions of each other.

TABLE IV  
SIMULATION OF ALL QUERIES

| Query          | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $Send(Q, m)$   | On a query $Send(\Pi_{DR_i}^x, \text{start})$ , assume that $\Pi_{DR_i}^x$ performs the following operations in the normal state. Generate $n_1$ and $T_1$ , decrypt $E_i$ to get $DM_i$ , and compute $D_i, F_i$ , and $Auth_1$ . Then, the query returns $Msg_1 = \{D_i, F_i, Auth_1, MID_j, T_1\}$ .                                                                                                                                                                                                                                           |
|                | On a query $Send(\Pi_{MR_j}^y, (D_i, F_i, Auth_1, MID_j, T_1))$ , assume $\Pi_{MR_j}^y$ is a normal state. $\Pi_{MR_j}^y$ computes $r_2, M_i, DM_i, n_1$ , and $Auth_1^*$ and checks $Auth_1^*$ . If the verification holds, it selects $n_2, T_2$ , then computes $G_i, SK_1$ , and $Auth_2$ . The query is answered by $Msg_2 = \{Auth_2, G_i, T_2\}$ .                                                                                                                                                                                         |
|                | On a query $Send(\Pi_{DR_i}^x, (Auth_2, G_i, T_2))$ , suppose $\Pi_{DR_i}^x$ computes $n_2, SK_1$ , and $Auth_2^*$ in a normal state and verifies $Auth_2^*$ . If it is not equal, the query process will be terminated. Otherwise, $\Pi_{DR_i}^x$ accepts and terminates.                                                                                                                                                                                                                                                                        |
|                | On a query $Send(\Pi_{MR_j}^y, \text{start})$ , assume that the state of $\Pi_{MR_j}^y$ is the normal state. $\Pi_{MR_j}^y$ computes $r_2, (C_i \parallel P_i), a_4$ , and $HC_i$ . Subsequently, $\Pi_{MR_j}^y$ selects $n_3$ and $T_3$ and computes $MN_i, Auth_3$ . Then, the query returns the output $Msg_3 = \{MN_i, T_3, Auth_3, CID_k\}$ .                                                                                                                                                                                                |
|                | On a query $Send(\Pi_{CS_k}^z, (MN_i, T_3, Auth_3, CID_k))$ , suppose $\Pi_{CS_k}^z$ is a normal state. $\Pi_{CS_k}^z$ computes $C_i, a_4$ , and $Auth_3^*$ and checks $Auth_3^*$ . $\Pi_{CS_k}^z$ calculates $HC_i$ and $n_3$ if $Auth_3^*$ is equal and generates $n_4$ , and $T_4$ . $\Pi_{CS_k}^z$ computes $CN_i, SK_2$ , and $Auth_4$ based on the calculated parameters. Next, $\Pi_{CS_k}^z$ returns $Msg_4 = \{CN_i, Auth_4, T_4\}$ .                                                                                                    |
|                | On a query $Send(\Pi_{MR_j}^y, (CN_i, Auth_4, T_4))$ , it is assumed that $\Pi_{MR_j}^y$ is in a normal state and performs some operations. After receiving the $Send$ query message $Msg_4 = \{CN_i, Auth_4, T_4\}$ , $\Pi_{MR_j}^y$ calculates $n_4, SK_2$ , and $Auth_4^*$ and then checks $Auth_4^*$ . If $Auth_4^*$ is incorrect, the query process is terminated. Finally, $\Pi_{MR_j}^y$ accepts and terminates.                                                                                                                           |
| $Execute(Q)$   | For an $Execute$ query, we use the simulation of $Send$ query as follows: $(D_i, F_i, Auth_1, MID_j, T_1) \leftarrow Send(\Pi_{DR_i}^x, \text{start})$ , $(Auth_2, G_i, T_2) \leftarrow Send(\Pi_{MR_j}^y, (D_i, F_i, Auth_1, MID_j, T_1))$ , $(MN_i, T_3, Auth_3, CID_k) \leftarrow Send(\Pi_{MR_j}^y, \text{start})$ , $(CN_i, Auth_4, T_4) \leftarrow Send(\Pi_{CS_k}^z, (MN_i, T_3, Auth_3, CID_k))$ . The query returns $(D_i, F_i, Auth_1, MID_j, T_1)$ , $(Auth_2, G_i, T_2)$ , $(MN_i, T_3, Auth_3, CID_k)$ , and $(CN_i, Auth_4, T_4)$ . |
| $Hash(string)$ | On a $Hash(string)$ query, if a record $(string, r)$ already exists in this query list, then $r = h(string)$ is returned. Else, select $r$ randomly, insert $(string, r)$ into the list, and return $r$ .                                                                                                                                                                                                                                                                                                                                         |
| $Corrupt(Q)$   | On a $Corrupt(\Pi_{DR_i}^x)$ query, if $\Pi_{DR_i}^x$ is accepted, the query is answered by parameters $\{MID_j, PID_c, E_i, DH_i, \tau_i, r_3, r_4\}$ in the smart device.                                                                                                                                                                                                                                                                                                                                                                       |
| $Test(Q)$      | On a $Test$ query, to flip the coin $c$ , if $c = 1$ , return $SK$ . Otherwise, return a random string of equal length to $SK$ .                                                                                                                                                                                                                                                                                                                                                                                                                  |

**Freshness:** An instance  $Q$  is considered fresh if it is in an acceptable state and the  $SK$  established between communicating entities is unknown to the adversary  $\mathcal{A}$ .

**Semantic Security of  $SK$ :** The semantic security of  $SK$  is determined by the ability of adversary  $\mathcal{A}$  to successfully guess its value through various queries. The challenge for  $\mathcal{A}$  lies in discriminating between the actual  $SK$  and a random string.

By employing these definitions, we can evaluate the semantic security of the  $SK$  in the proposed protocol.

### 3) Security Proof:

**Theorem 1:** Let  $\mathcal{A}$  be an adversary capable of executing the aforementioned queries and attacking our protocol  $P$  within probabilistic polynomial time, aiming to break the protocol's semantic security. The advantage of  $\mathcal{A}$  in breaking  $P$  is denoted as  $Adv_{\mathcal{A}}^P(\xi) \leq q_h^2/|Hash| + 2\{\max\{C \cdot q_s', q_s/2^l\} + Adv_{\Omega}^{IND-CPA}(k)\}$ , where  $q_h$  represents the number of executed  $Hash$  queries,  $q_s$  denotes the number of executed  $Send$  queries,  $|Hash|$  corresponds to the range space of the hash function  $h(\cdot)$ ,  $l$  indicates the bit length of the random biometric key.  $C$  and  $s'$  are constant parameters in Zipf's law [52]. The advantage of  $\mathcal{A}$  in breaking the IND-CPA symmetric cipher  $\Omega$  [53] is denoted as  $Adv_{\Omega}^{IND-CPA}(k) = Adv_{\Omega, SE}^{IND-CPA}(k)$  or  $Adv_{\Omega, ME}^{IND-CPA}(k)$ .

**Proof:** We construct a sequence of games from  $GM_0$  to  $GM_4$ , involving participants  $Q$  and  $\mathcal{A}$ . We formally prove the

protocol's security by analyzing these games. Throughout the proof,  $Succ_{\mathcal{A}}^{GM_i}$  represents the probability of  $\mathcal{A}$  successfully guessing the event of coin  $c$  in game  $GM_i$ .  $Adv_{\mathcal{A}}^P(\xi)$  denotes the advantage of  $\mathcal{A}$  in effectively breaking the protocol's security. The simulation of  $\mathcal{A}$ 's queries is outlined in Table IV. The flow of the games is as follows.

**$GM_0$ :** In the first round,  $\mathcal{A}$  does not initiate any query operation and only starts the game by flipping a coin  $c$ . We can obtain

$$Adv_{\mathcal{A}}^P(\xi) = \left| 2Pr[Succ_{\mathcal{A}}^{GM_0}] - 1 \right|. \quad (1)$$

**$GM_1$ :** In the second round of the game,  $\mathcal{A}$  executes the  $Execute$  query to simulate an eavesdropping attack, allowing  $\mathcal{A}$  to eavesdrop on the transmission of  $\{Msg_1, Msg_2, Msg_3, Msg_4, Msg_5, Msg_6\}$  over the public channel. At the end of  $GM_1$ , based on the obtained parameters,  $\mathcal{A}$  uses the  $Test$  query to derive  $SK_1$  and  $SK_2$ . However, since  $\mathcal{A}$  is unable to capture  $\{DM_i, n_1, n_2, C_i, n_3, n_4\}$ , the probability of  $\mathcal{A}$  winning the current round does not increase compared to the previous round. Therefore, the likelihood of  $GM_1$  is equal to  $GM_0$ , which can be expressed as:

$$Pr[Succ_{\mathcal{A}}^{GM_1}] = Pr[Succ_{\mathcal{A}}^{GM_0}]. \quad (2)$$

**$GM_2$ :** In  $GM_2$ , an active attack is introduced, distinguishing it from  $GM_1$ , by including the simulation of  $Send$  and  $Hash$



queries. In this game, the goal of  $\mathcal{A}$  is to deceive the participants by sending a modified fake message and creating the illusion of legitimate news.  $\mathcal{A}$  continuously executes Hash queries to verify if there are any hash collisions. Note that although  $\{Msg_1, Msg_2, Msg_3, Msg_4, Msg_5, Msg_6\}$  are messages exchanged publicly, their composition directly or indirectly contains dynamic and random parameters, such as random numbers, timestamps, identities, and long-term secrets. As a result, when  $\mathcal{A}$  performs the *Send* query, no hash collisions occur. By applying the birthday paradox, we can deduce that

$$\left| \Pr[\text{Succ}_{\mathcal{A}}^{GM_2}] - \Pr[\text{Succ}_{\mathcal{A}}^{GM_1}] \right| \leq \frac{q_h^2}{2|\text{Hash}|}. \quad (3)$$

$GM_3$ : In this round,  $\mathcal{A}$  utilizes the *Corrupt* query to simulate the game. Through power analysis,  $\mathcal{A}$  can extract sensitive information stored in the smart device of  $DR_i$ , including  $\{MID_j, PID_c, E_i, DH_i, \tau_i, r_3, r_4\}$ . As an illegal insider,  $\mathcal{A}$  has access to the data stored inside the cloud server, including  $\{PID_c, CID_k, S_i, Q_i\}$ . In our protocol,  $DR_i$  employs both a password  $PW_i$  and biometric information  $Bio_i$ . Since users often choose passwords with low entropy,  $\mathcal{A}$  may attempt an offline password-guessing attack to extract user passwords. The probability of  $\mathcal{A}$  successfully guessing  $PW_i$  depends on whether the biometric random key  $\delta_i$  of  $Bio_i$  is known. In the case of  $\delta_i \in \{0, 1\}^l$ , the probability of guessing a biometric random key  $\delta_i$  of  $l$  bits in length is approximately  $1/2^l$ . The system imposes a limit on the number of incorrect password attempts. If the offline password-guessing attack is unsuccessful,  $GM_2$  and  $GM_3$  become similar. With the inclusion of Zipf's law [52], we obtain

$$\left| \Pr[\text{Succ}_{\mathcal{A}}^{GM_3}] - \Pr[\text{Succ}_{\mathcal{A}}^{GM_2}] \right| \leq \max \left\{ C \cdot q_s', \frac{q_s}{2^l} \right\}. \quad (4)$$

$GM_4$ : In the final round,  $GM_4$ ,  $\mathcal{A}$  attempts to decrypt  $E_i$  and  $O_i$  using the intercepted  $\{Msg_1, Msg_2, Msg_3, Msg_4, Msg_5, Msg_6\}$  in order to crack the session keys  $SK_1$  and  $SK_2$ . The decryption of  $E_i$  requires the corresponding key, and  $DM_i$  is a crucial parameter in the computation of  $SK_1$ , which  $\mathcal{A}$  can only obtain if it successfully decrypts  $E_i$ . Similarly, for the computation of  $SK_2$  using  $C_i$ ,  $\mathcal{A}$  needs to decrypt  $O_i$ . The encryption and decryption operations involving these parameters utilize the computationally expensive AES-128 encryption/decryption in CBC mode [53], with a random initialization vector (V) used for each operation. Considering the IND-CPA property, we have

$$\left| \Pr[\text{Succ}_{\mathcal{A}}^{GM_4}] - \Pr[\text{Succ}_{\mathcal{A}}^{GM_3}] \right| \leq Adv_{\Omega}^{IND-CPA}(k). \quad (5)$$

After completing all the games,  $\mathcal{A}$  randomly guesses the bit  $c$  using the *Test* query. Thus, we have

$$\Pr[\text{Succ}_{\mathcal{A}}^{GM_4}] = \frac{1}{2}. \quad (6)$$

Based on the above equations, we can derive

$$\begin{aligned} \frac{1}{2} Adv_{\mathcal{A}}^{\mathcal{P}}(\xi) &= \left| \Pr[\text{Succ}_{\mathcal{A}}^{GM_0}] - \frac{1}{2} \right| \\ &= \left| \Pr[\text{Succ}_{\mathcal{A}}^{GM_1}] - \Pr[\text{Succ}_{\mathcal{A}}^{GM_4}] \right| \end{aligned}$$

$$\begin{aligned} &\leq \left| \Pr[\text{Succ}_{\mathcal{A}}^{GM_1}] - \Pr[\text{Succ}_{\mathcal{A}}^{GM_2}] \right| \\ &\quad + \left| \Pr[\text{Succ}_{\mathcal{A}}^{GM_2}] - \Pr[\text{Succ}_{\mathcal{A}}^{GM_3}] \right| \\ &\quad + \left| \Pr[\text{Succ}_{\mathcal{A}}^{GM_3}] - \Pr[\text{Succ}_{\mathcal{A}}^{GM_4}] \right| \\ &\leq \frac{q_h^2}{2|\text{Hash}|} + \max \left\{ C \cdot q_s', \frac{q_s}{2^l} \right\} + Adv_{\Omega}^{IND-CPA}(k). \end{aligned} \quad (7)$$

Thus, we can conclude

$$Adv_{\mathcal{A}}^{\mathcal{P}}(\xi) \leq \frac{q_h^2}{|\text{Hash}|} + 2 \left\{ \max \left\{ C \cdot q_s', \frac{q_s}{2^l} \right\} + Adv_{\Omega}^{IND-CPA}(k) \right\}. \quad (8)$$

■

## B. Security Requirements

In this part, we perform the informal security analysis to demonstrate that our proposed protocol can meet standard security requirements.

1) *Mutual Authentication*: The proposed protocol achieves mutual authentication between the relevant communication entities. Authentication values, such as  $Auth_1$  in  $Msg_1$  and  $Auth_2$  in  $Msg_2$  are used to authenticate the identities of  $DR_i$  and  $MR_j$ , respectively. Similarly,  $Auth_3$  in  $Msg_3$  and  $Auth_4$  in  $Msg_4$  authenticate the identities of  $MR_j$  and  $CS_k$ , respectively, while  $Auth_5$  in  $Msg_5$  and  $Auth_6$  in  $Msg_6$  authenticate the identities of  $MR_j$  and  $PT_c$ , respectively. Therefore, mutual authentication is achieved between  $DR_i$  and  $MR_j$ ,  $MR_j$  and  $CS_k$ , and  $MR_j$  and  $PT_c$ .

2) *Data Tampering Attack*: The protocol we have formulated incorporates a mutual authentication process, wherein the two communicating entities undergo identity authentication. This involves verifying the legitimacy of each other's identities during communication. Only through successful authentication is the session established, facilitating the subsequent establishment of a SK for the secure transmission of critical data. Additionally,  $\mathcal{A}$  is restricted to accessing messages in the public channel or data stored by a specific entity. Importantly,  $\mathcal{A}$  is unable to exploit tampered data to subvert identity authentication or compute the SK. Consequently, our protocol remains resilient against data tampering attacks.

3) *Stolen Smart Device Attack*: In the event of a stolen smart device attack, where  $\mathcal{A}$  gains access to the data  $\{MID_j, PID_c, E_i, DH_i, \tau_i, r_3, r_4\}$  stored in  $DR_i$ 's device, the lack of  $DP_i$  prevents  $\mathcal{A}$  from decrypting  $E_i$  and obtaining  $DM_i$ . Without  $DM_i$ ,  $\mathcal{A}$  cannot calculate the random numbers  $n_1$  and  $n_2$  using  $n_1 = h(D_i \| DM_i) \oplus F_i$  and  $n_2 = h(n_1 \| DM_i) \oplus G_i$ . Consequently,  $\mathcal{A}$  cannot compute the session key  $SK_1 = h(DM_i \| n_1 \| n_2)$ , preventing it from breaking  $SK_1$ . Thus, the protocol effectively resists stolen smart device attacks.

4) *Identity Theft Attack*: Suppose  $\mathcal{A}$  successfully acquires the identity value ID of  $DR_i$  and intercepts the data transmitted in the public channel. Upon attempting to compute  $DP_i = h(ID_i \| r_3) \oplus h(PW_i^* \| \delta_i^*)$ ,  $\mathcal{A}$  faces the challenge that  $r_3$  and the password  $PW_i^*$  remain unknown. Additionally,  $\mathcal{A}$  lacks access to the biometric value of  $DR_i$ , further hindering the computation of  $DP_i^*$ . Given the unknown  $DP_i^*$ ,  $\mathcal{A}$  encounters

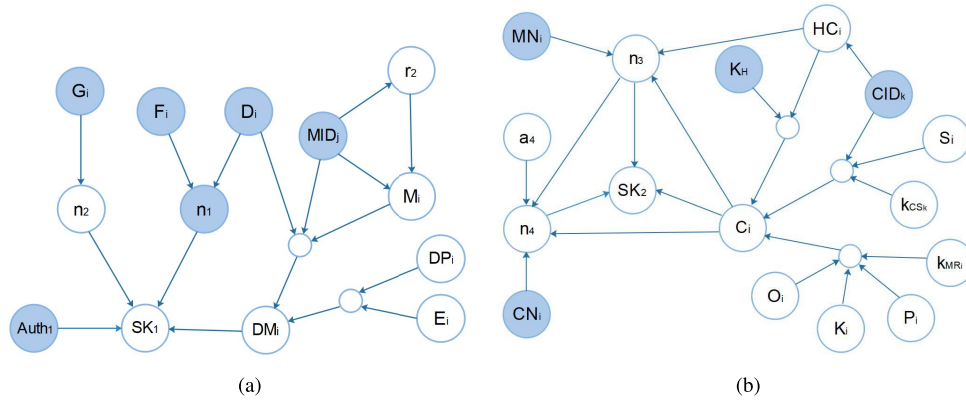


Fig. 7. Final result for perfect forward secrecy. (a) Directed graph for session key  $SK_1$ . (b) Directed graph for session key  $SK_2$ .

difficulty in calculating  $DH_i$ , resulting in a login failure. Importantly,  $\mathcal{A}$  is unable to derive the session key  $SK_1$ . Consequently, our protocol effectively mitigates identity theft attacks.

5) *Replay Attack*: The protocol employs timestamps  $\{T_1, T_2, T_3, T_4, T_5, T_6\}$  to prevent replay attacks. For instance, when  $MR_j$  receives  $Msg_1$  from  $DR_i$ , it verifies the freshness of  $T_1$  before proceeding with further processing. If  $T_1$  is invalid, the protocol is interrupted. Even if  $\mathcal{A}$  manages to intercept and replay  $Msg_1$  multiple times, the invalidity of  $T_1$  will cause the authentication to fail, halting the session process. Therefore, the protocol provides resistance against replay attacks.

6) *Privileged Insider Attack*: In the scenario where  $\mathcal{A}$  is an insider with access to the cloud server  $CS_k$ , it gains knowledge of all the stored data ( $\{PID_c, CID_k, S_i, Q_i\}$ ). However, without knowledge of the private key  $k_{CS_k}$ ,  $\mathcal{A}$  cannot compute  $C_i = S_i \oplus h(CID_k \| k_{CS_k})$ . The inability to determine  $C_i$  prevents  $\mathcal{A}$  from calculating  $n_3$  and  $n_4$ , ensuring the security of the negotiated session key  $SK_2$ . Hence, the protocol is resilient against privileged insider attacks.

7) *Offline Password-Guessing Attack*: The protocol is designed to mitigate offline password-guessing attacks. Even if  $\mathcal{A}$  obtains  $DR_i$ 's login password, it cannot determine the biometric random key  $\delta_i$  since it is derived from the unique and secret biometric information  $Bio_i$ . Additionally,  $\mathcal{A}$  faces difficulty in guessing  $DR_i$ 's identity while attempting to guess the password and analyzing the saved random number  $r_3$  in  $DR_i$ 's smart device. Insufficient parameters prevent  $\mathcal{A}$  from computing  $DP_i = h(ID_i \| r_3) \oplus h(PW_i \| \delta_i)$  and  $DH_i = h(DP_i \| \delta_i \| ID_i)$ . As a result,  $\mathcal{A}$  fails to compute  $DH_i$  during the login phase, preventing successful login. Therefore, the protocol remains secure against offline password-guessing attacks.

8) *Man-in-The-Middle Attack*:  $\mathcal{A}$  intercepts  $D_i, F_i, Auth_1, MID_j, T_1$  on the public channel and attempts to impersonate  $MR_j$  in communication with  $DR_i$ . However, due to the lack of knowledge regarding  $K_i$  and  $k_{MR_j}$ ,  $\mathcal{A}$  generates its own  $K'_i$  and  $k'_{MR_j}$  for impersonation. The adversary calculates  $r'_2 = K'_i \oplus h(MID_j \| k'_{MR_j})$ ,  $M'_i = h(MID_j \| r'_2)$ ,  $DM'_i = h(D_i \| MID_j \| M'_i)$ ,  $n'_1 = h(D_i \| DM'_i) \oplus F_i$ , and  $Auth'_2 = h(DM'_i \| Auth_1 \| SK'_1)$  successively.  $DR_i$  computes  $SK_1 = h(DM_i \| n_1 \| n_2)$  and

$Auth'_2 = h(DM_i \| Auth_1 \| SK_1)$  upon receiving  $Auth'_2, G'_i, T_2$ . Due to  $DM_i \neq DM'_i$  and  $n_1 \neq n'_1$ , the computed  $Auth'_2 \neq Auth_2$ . Consequently,  $DR_i$  fails to authenticate  $\mathcal{A}$ . A similar scenario unfolds when  $\mathcal{A}$  attempts to impersonate  $DR_i$  in communication with  $MR_j$ . This affirms the protocol's resilience against man-in-the-middle attacks.

9) *Perfect Forward Secrecy*: Suppose an adversary  $\mathcal{A}$  obtains the long-term key  $K_H$  of  $HP$  and receives information  $\{PID_c, T_5, T_6\}$  through the public channel. However, the random numbers  $a_1$  and  $a_2$  are only stored in  $HP$ 's database and are not accessible to  $\mathcal{A}$ . As a result,  $\mathcal{A}$  cannot compute  $A_i = h(PID_c \| K_H \| a_2)$  and  $B_i = h(A_i \| a_1) \oplus PID_c$  by calculation. The absence of  $A_i$  and  $B_i$  prevents  $\mathcal{A}$  from forging authentication values  $Auth_5$  and  $Auth_6$ , thereby preserving the mutual authentication between  $MR_j$  and  $PT_c$ . Furthermore, since  $\mathcal{A}$  does not have access to  $a_4$  stored by  $HP$ , it cannot compute  $HC_i$  and  $C_i$ . The random numbers  $n_3$  and  $n_4$  required to calculate session key  $SK_2$  are not transmitted through the public channel, thus preventing  $\mathcal{A}$  from deducing  $SK_2$ . Similarly, due to the security of the critical parameter  $DM_i$ ,  $\mathcal{A}$  is unable to break  $SK_1$ .

Moreover, employing the method proposed by Ge et al. [54], we can demonstrate that the proposed protocol ensures perfect forward secrecy. By constructing directed graphs representing the session keys  $SK_1$  and  $SK_2$ , we observe from Fig. 7(a) and (b) that the nodes  $\{Auth_1, DM_i, n_1, n_2\}$  and the nodes  $\{C_i, n_3, n_4\}$  are never simultaneously colored. This observation indicates that even if  $\mathcal{A}$  gains access to  $K_H$ , it is still unable to compute session keys  $SK_1$  and  $SK_2$ . Hence, our protocol provides perfect forward secrecy.

In conclusion, the protocol remains secure even if  $\mathcal{A}$  obtains  $K_H$ . It ensures the authenticity and mutual authentication between  $MR_j$  and  $PT_c$ , prevents the computation of session keys  $SK_1$  and  $SK_2$  by  $\mathcal{A}$ , and provides perfect forward secrecy.

### C. Security Comparisons

This section provides a comprehensive security comparison of the protocols proposed by Jia et al. [23], Ma et al. [55], Wu et al. [56], and Li et al. [57]. The specific results of the comparison are presented in Table V. In the table, a  $\checkmark$  denotes that the protocol meets the security requirements,  $\times$  indicates that the protocol fails to fulfill the security requirements, and

TABLE V  
COMPARISONS OF SECURITY

| Security requirements                                      | Jia et al. [23] | Ma et al. [55] | Wu et al. [56] | Li et al. [57] | Ours |
|------------------------------------------------------------|-----------------|----------------|----------------|----------------|------|
| Resist known session-specific temporary information attack | × [24]          | × [58]         | ✓              | ✓              | ✓    |
| Ensure anonymity and untraceability                        | × [24]          | × [58], [59]   | ✓              | ✓              | ✓    |
| Resist impersonation attack                                | × [26]          | × [59]         | ✓              | -              | ✓    |
| Resist privileged insider attack                           | -               | × [58]         | × [60]         | -              | ✓    |
| Resist stolen smart device attack                          | ✓               | × [58]         | × [60]         | ✓              | ✓    |
| Resist de-synchronization attack                           | -               | × [58]         | -              | -              | ✓    |
| Ensure perfect forward secrecy                             | ✓               | ✓              | × [60]         | ✓              | ✓    |
| Resist replay attack                                       | ✓               | ✓              | ✓              | ✓              | ✓    |
| Resist offline password guessing attack                    | ✓               | ✓              | ✓              | ✓              | ✓    |
| Ensure mutual authentication                               | -               | ✓              | ✓              | -              | ✓    |
| Resist stolen-verifier attack                              | ✓               | × [58]         | -              | ✓              | ✓    |

TABLE VI  
PARAMETERS OF THE DEVICES

| Devices                 | CPU                                  | Operating system | Running memory |
|-------------------------|--------------------------------------|------------------|----------------|
| Smartisan Pro 2         | Qualcomm Technologies                | Android system   | 4GB            |
| Dell Laptop Computer    | Intel(R) Core(TM)i5-7200U CPU@2.5GHz | Windows 10       | 8GB            |
| Lenovo Desktop Computer | Intel(R) Core(TM)i5-9500 CPU@3.00GHz | Windows 10       | 8GB            |

TABLE VII  
EXECUTION TIME OF OPERATIONS

| Operations                                  | Smartisan Pro 2 | Dell Laptop Computer | Lenovo Desktop Computer |
|---------------------------------------------|-----------------|----------------------|-------------------------|
| Bilinear Pairing( $T_{bp}$ )                | 24.5691ms       | 13.4228ms            | 5.4973ms                |
| Symmetric Encryption/Decryption( $T_{ed}$ ) | 7.9683ms        | 3.6915ms             | 1.6877ms                |
| Scalar Multiplication( $T_{sm}$ )           | 9.6305ms        | 4.8847ms             | 2.3764ms                |
| Hash Function( $T_h$ )                      | 0.0062ms        | 0.0054ms             | 0.0031ms                |
| Fuzzy Extraction Function( $T_{fe}$ )       | 0.6479ms        | 0.4723ms             | 0.3826ms                |

- indicates the absence of information regarding the protocol's compliance with the security requirements.

As indicated in Table V, Jia et al.'s protocol [23] is vulnerable to known session-specific temporary information and impersonation attacks. Moreover, it fails to ensure user anonymity and untraceability, as discussed in [24] and [26]. Ma et al.'s protocol [55] is susceptible to known session-specific temporary information attacks, privileged insider attacks, de-synchronization attacks, and other security concerns, as shown in more detail in [58] and [59]. Additionally, it does not provide user anonymity and untraceability. Furthermore, Li et al. [60] have pointed out that Wu et al.'s protocol [56] is insecure, prone to privileged insider attacks and stolen smart device attacks, and lacks perfect forward secrecy. In comparison to the aforementioned protocols [23], [55], [56], Li et al.'s [57] protocol, along with our proposed protocol, demonstrates resilience against various common attacks and exhibits high levels of security.

## VI. PERFORMANCE COMPARISONS

This section undertakes a rigorous comparative analysis between the proposed protocol and those advanced by Jia et al. [23], Ma et al. [55], Wu et al. [56], and Li et al. [57]. The comparison encompasses the communication and computational costs associated with these protocols.

### A. Test Environment Setup

This section examines the computational and communication costs to evaluate the efficiency of the protocols. The evaluation targets explicitly the mutual authentication and SK establishment phases. To assess the performance, we conduct simulation experiments using specific devices, namely, the Smartisan Pro 2, to simulate the doctor/user/vehicle, Dell Laptop Computer to simulate the medical robot/fog node, and Lenovo Desktop Computer to simulate the cloud server. Average execution times, obtained from 10 runs on each device, are presented in Table VI for experimental parameters and Table VII for execution times.

In terms of computational cost, operations, such as the bilinear pairing function ( $T_{bp}$ ), symmetric encryption/decryption operation ( $T_{ed}$ ), scalar multiplication ( $T_{sm}$ ), hash function ( $T_h$ ), and fuzzy extraction function ( $T_{fe}$ ), are considered. The computational cost of  $\parallel$  and  $\oplus$  operations is deemed negligible.

### B. Computational Cost Comparisons

Following the data presented in Table VIII, the execution times for Jia et al.'s protocol [23] and Ma et al.'s protocol [55] are 79.7293ms and 72.2748ms, respectively. Conversely, Wu et al.'s protocol [56] demonstrates the lowest computational cost, recording only 0.1045ms. It is noteworthy, however, that Wu et al.'s protocol [56] is susceptible to security vulnerabilities, particularly in its resilience against

TABLE VIII  
COMPUTATIONAL COST COMPARISON

| Protocols       | Doctor/User/Vehicle(ms)                   | Medical Robot/Fog Node(ms)                | Cloud Server(ms)                          | Total(ms)                                   |
|-----------------|-------------------------------------------|-------------------------------------------|-------------------------------------------|---------------------------------------------|
| Jia et al. [23] | $5T_h + T_{bp} + 2T_{sm} \approx 43.8611$ | $4T_h + T_{bp} + 2T_{sm} \approx 23.2138$ | $9T_h + T_{bp} + 3T_{sm} \approx 12.6544$ | $18T_h + 3T_{bp} + 7T_{sm} \approx 79.7293$ |
| Ma et al. [55]  | $4T_h + 3T_{sm} \approx 28.9163$          | $4T_h + 4T_{sm} \approx 19.5604$          | $11T_h + 10T_{sm} \approx 23.7981$        | $19T_h + 17T_{sm} \approx 72.2748$          |
| Wu et al. [56]  | $7T_h \approx 0.0434$                     | $5T_h \approx 0.027$                      | $11T_h \approx 0.0341$                    | $23T_h \approx 0.1045$                      |
| Li et al. [57]  | $7T_h + 4T_{sm} \approx 38.5654$          | $5T_h + 5T_{sm} \approx 24.4505$          | $12T_h + 6T_{sm} \approx 14.2956$         | $24T_h + 15T_{sm} \approx 77.3115$          |
| Ours            | $9T_h + T_{fe} + Ted \approx 8.6720$      | $15T_h + Ted \approx 3.7725$              | $8T_h \approx 0.0248$                     | $32T_h + T_{fe} + 2Ted \approx 12.4693$     |

TABLE IX  
COMMUNICATION COST COMPARISON

| Protocols       | Rounds | Doctor/User/Vehicle(bits) | Medical Robot/Fog Node(bits) | Cloud Server(bits) | Total(bits)                           |
|-----------------|--------|---------------------------|------------------------------|--------------------|---------------------------------------|
| Jia et al. [23] | 4      | 768                       | 2464                         | 864                | $6 h  + 6 sm  + 3 id  + 5 t  = 4096$  |
| Ma et al. [55]  | 4      | 768                       | 3104                         | 1504               | $6 h  + 10 sm  + 3 id  + 5 t  = 5376$ |
| Wu et al. [56]  | 4      | 640                       | 1312                         | 832                | $4 h  + 7 n  + 4 id  = 2784$          |
| Li et al. [57]  | 4      | 1088                      | 3744                         | 1504               | $6 h  + 13 sm  + 3 id  + 5 t  = 6336$ |
| Ours            | 4      | 864                       | 1056                         | 448544             | $5 h  + 3 n  + 3 id  + 4 t  = 2368$   |

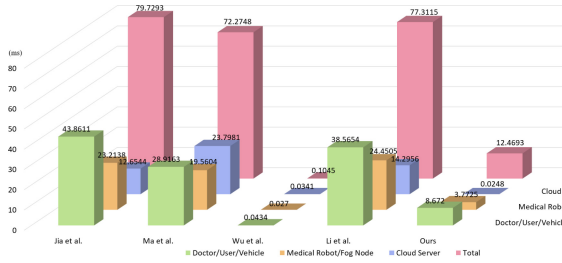


Fig. 8. Computational cost.

privileged insider attacks and stolen smart device attacks, thereby compromising perfect forward secrecy. Furthermore, Li et al.'s protocol [57] is deemed secure but exhibits a prolonged execution time of 77.3115ms compared to our proposed protocol. The visual representation of the comparative analysis of execution times for each protocol is depicted in Fig. 8.

We also systematically vary the number of entities to assess the scalability of different protocols in the face of increasing entities. Specifically, we increment the number of doctors/users/vehicles, the number of medical robots/fog nodes ranging from 5 to 25, and the number of cloud servers from 1 to 5. The computational costs are graphically represented in Fig. 9(a)–(c), respectively. It indicates that the computational cost of our protocol exhibits reasonable variation, maintaining smooth and superior performance amidst changes in the number of entities.

Upon a comprehensive comparison, the proposed protocol showcases commendable performance concerning computational cost. In this regard, our protocol surpasses the computational costs of Jia et al.'s [23], Ma et al.'s [55], and Li et al.'s [57] protocols while marginally exceeding that of Wu et al.'s [56] protocol. However, it is imperative to acknowledge that Wu et al.'s protocol [56] lacks resilience against well-known attacks.

### C. Communication Cost Comparisons

In evaluating communication overhead, specific lengths are assumed for various parameters within the system. Notably, these include the hash function ( $|h| = 256$  bits), timestamp ( $|t| = 32$  bits), random number ( $|n| = 160$  bits), scalar

multiplication ( $|sm| = 320$  bits), and identity ( $|id| = 160$  bits). Taking our proposed protocol as an illustrative example, the messages transmitted through the public channel are defined as  $Msg_1 = D_i, F_i, Auth_1, MID_j, T_1$ ,  $Msg_2 = Auth_2, G_i, T_2$ ,  $Msg_3 = MN_i, Auth_3, CID_k, T_3$ , and  $Msg_4 = CN_i, Auth_4, T_4$ . The lengths of identities, random numbers, hash functions, and timestamps are explicitly specified. Consequently, the communication cost of our proposed protocol is determined as  $5|h| + 3|n| + 3|id| + 4|t| = 2368$  bits. Similar calculations are undertaken to ascertain the communication costs of Jia et al.'s protocol [23], Ma et al.'s protocol [55], Wu et al.'s protocol [56], Li et al.'s protocol [57], and the proposed protocol.

As depicted in Table IX, Jia et al.'s protocol [23] incurs a communication cost of  $6|h| + 6|sm| + 3|id| + 5|t| = 4096$  bits, Ma et al.'s protocol [55] registers a communication cost of  $6|h| + 10|sm| + 3|id| + 5|t| = 5376$  bits, and Wu et al.'s protocol [56] results in a communication cost of  $4|h| + 7|n| + 4|id| = 2784$  bits. Despite the security of Li et al.'s protocol [57], it exhibits the highest communication overhead. Conversely, our proposed protocol demonstrates the lowest communication overhead at 2368 bits.

In Fig. 10, the visual comparison of communication costs among all protocols [23], [55], [56], [57] reveals that our proposed protocol exhibits the least computational overhead and communication cost when compared with other relevant protocols. This comparison underscores the significant advantage of our proposed protocol in terms of communication cost.

### D. Blockchain Implementation

The platform configuration employed in the blockchain simulation comprised Ubuntu 20.04, an Intel Core i7-11800H@2.30-GHz processor, a 64-bit operating system, and 2 GB of RAM. Additionally, Visual Studio Code version 1.75 was utilized for development purposes [61].

During the execution of the simulation, the quantity of transactions undergoes an incremental phase, and upon reaching a predefined threshold, a leader  $L$  is democratically elected within the P2P CS network. Subsequently, the verified blocks are uploaded to the blockchain by applying the voting-based PBFT consensus algorithm, denoted as Algorithm 1. Following successfully completing the voting process, the



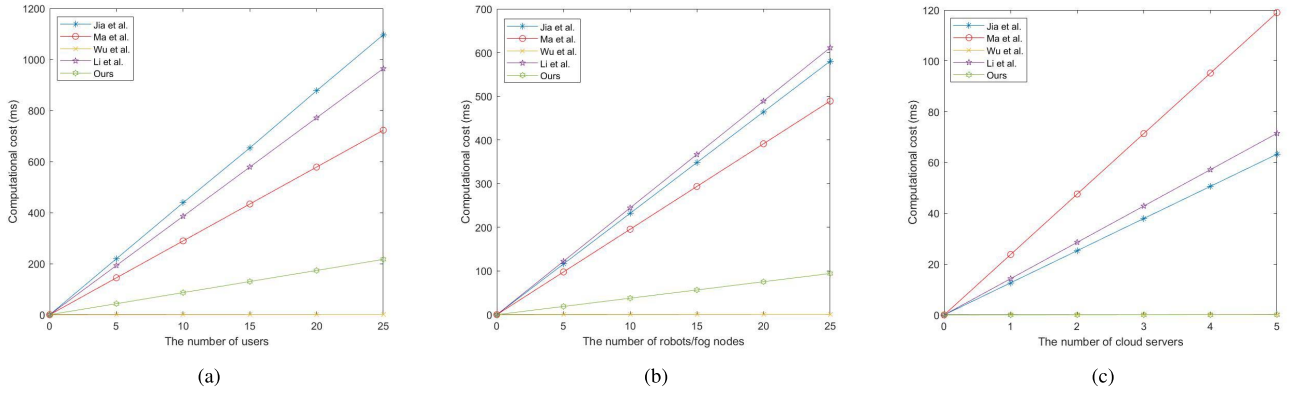


Fig. 9. Computational costs vary with the number of entities. (a) Computational costs vary with the number of doctors/users/vehicles. (b) Computational costs vary with the number of medical robots/fog nodes. (c) Computational costs vary with the number of cloud servers.

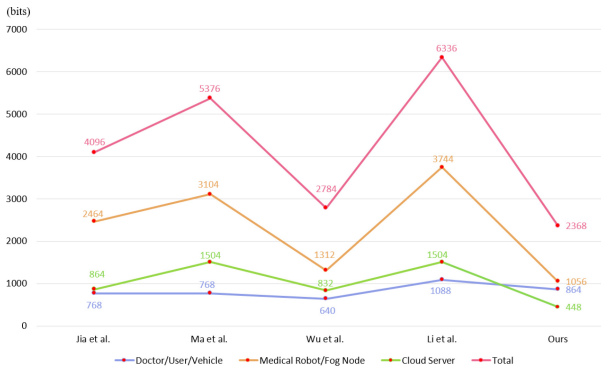


Fig. 10. Communication cost.

structured blocks, as delineated in Fig. 6, are integrated into the blockchain.

The determination of the block size, as illustrated in Fig. 6, involves considerations for the block version, hash of the previous block, Merkle tree root, current timestamp, block owner, owner's public key, block signature utilizing ECDSA, and the hash of the current block, with dimensions of 32, 256, 256, 32, 160, 320, 320, and 256 bits, respectively. Each transaction  $Tn_x$ , utilizing ECC as the encryption mechanism, generates two points, thus necessitating a transaction size of  $(320 + 320) = 640$  bits. Consequently, the payload size for the  $n_t$  encrypted transaction block is determined to be  $640 \times n_t$  bits. Computationally, the complete block size is established as  $1632 + (640 \times n_t)$  bits. The simulation assumes that the number of nodes in the P2P CS network is fixed at 5. The performance evaluation is conducted under the following scenarios.

- 1) *Scenario 1*: The simulation is executed with a constant value of 30 for the number of blocks, and the variation results are depicted in Fig. 11(a). The computation time demonstrates a linear escalation as the number of mined blocks increases.
- 2) *Scenario 2*: The simulation is conducted with a fixed value of 60 for the number of transactions in each block, and the experimental outcomes are presented in Fig. 11(b). Here, the computation time exhibits a linear increase with the growing number of mined blocks.

In summary, in both scenarios, the total computation time manifests a linear progression with the increase in the number of transactions within a block or the expansion of the number of mined blocks.

## VII. CONCLUSION

This article first delves into the application of wearable technology in healthcare, emphasizing its potential in redefining patient monitoring and data collection and recognizing the urgent need for data protection measures. Besides, through in-depth research, we found that cloud computing and blockchain have significant advantages in storing and managing private patient data, fully recognizing their positive role in establishing a secure, transparent, and tamper-proof framework for sensitive patient information. In addition, a comprehensive introduction to the operational framework of medical robots is presented, emphasizing their indispensable integration into the medical industry. However, protecting medical data involves storage and establishing robust mechanisms during data transmission and collaboration. After conducting in-depth research on authentication protocols designed for medical scenarios, we found that many protocols have security issues during data transmission. In response to identified security vulnerabilities during data transmission in medical scenarios, this article proposes a mutual authentication and key agreement protocol. This protocol enhances medical services with medical robots' support, ensuring patient data's confidentiality and integrity throughout its lifecycle. This article includes formal and informal security analyses to validate the protocol's security and correctness. Through a comparative evaluation of the protocol's performance, we substantiate its superiority in terms of security and efficiency.

Despite the substantial progress made by the proposed protocol in enhancing the security of healthcare services, certain limitations warrant careful consideration. The protocol's performance may be susceptible to challenges in extreme network conditions, such as high-latency or low-bandwidth environments. Furthermore, while our protocol accentuates the security of data transmission to patients and the utilization of medical robots for assisted treatment, the hardware security of medical robots remains a critical consideration in practical

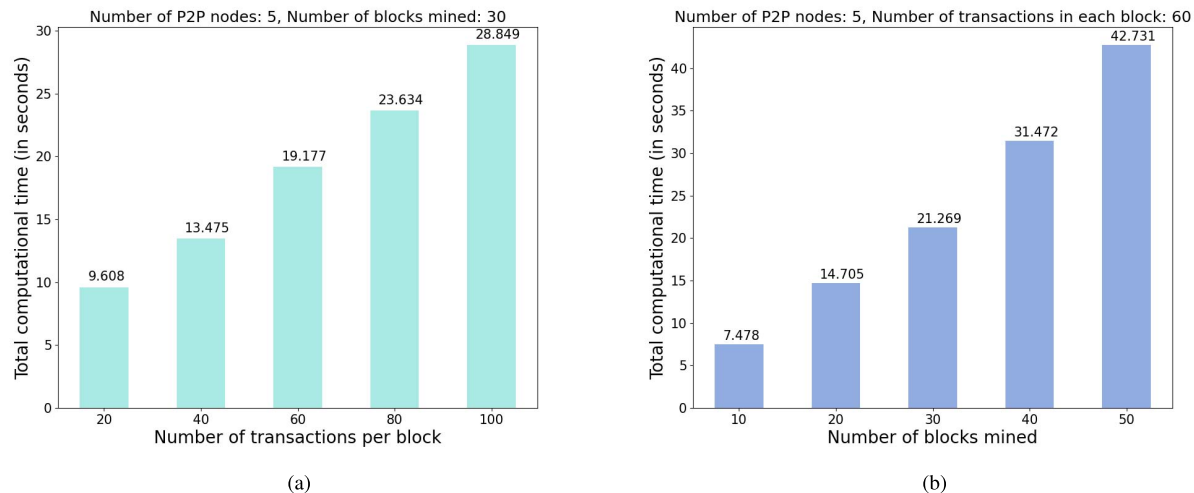


Fig. 11. Blockchain simulation results. (a) Scenario 1. (b) Scenario 2.

applications. Future endeavors should optimize the protocol to ensure robustness under diverse and complex network conditions and enhance hardware security to withstand potential physical attacks and malicious tampering. Additionally, scalability optimization is essential, accommodating the diversity of devices and operating systems in the medical environment, considering the mobile characteristics of medical robots and practical application scenarios.

## REFERENCES

- [1] S. Madakam, R. Ramaswamy, and S. Tripathi, "Internet of Things (IoT): A literature review," *J. Comput. Commun.*, vol. 3, no. 5, p. 164, 2015.
- [2] W. H. Hassan, "Current research on Internet of Things (IoT) security: A survey," *Comput. Netw.*, vol. 148, pp. 283–294, Jan. 2019.
- [3] S. S.-D. Xu, I. G. D. Pancev, and M. Y. Baihaqi, "Design and implementation of a 6lowpan-based lightweight wireless embedded Internet platform for IoT applications," *J. Internet Technol.*, vol. 24, no. 2, pp. 323–332, 2023.
- [4] S. Huhn et al., "The impact of wearable technologies in health research: Scoping review," *JMIR mHealth uHealth*, vol. 10, no. 1, 2022, Art. no. e34384.
- [5] X.-M. Loh, V.-H. Lee, G. W.-H. Tan, J.-J. Hew, and K.-B. Ooi, "Towards a cashless society: The imminent role of wearable technology," *J. Comput. Inf. Syst.*, vol. 62, no. 1, pp. 39–49, 2022.
- [6] C.-M. Chen, Z. Chen, A. K. Das, and S. A. Chaudhry, "A security-enhanced and ultra-lightweight communication protocol for Internet of Medical Things," *IEEE Internet Things J.*, vol. 11, no. 6, pp. 10168–10182, Mar. 2024.
- [7] S. O. Ajakwe, C. I. Nwakanma, D.-S. Kim, and J.-M. Lee, "Key wearable device technologies parameters for innovative healthcare delivery in B5G network: A review," *IEEE Access*, vol. 10, pp. 49956–49974, 2022.
- [8] B. Wang et al., "Wearable aptamer-field-effect transistor sensing system for noninvasive cortisol monitoring," *Sci. Adv.*, vol. 8, no. 1, 2022, Art. no. eabk0967.
- [9] P. Kumar, M. Rahman, S. Namasudra, and N. R. Moparthi, "Enhancing security of medical images using deep learning, chaotic map, and hash table," *Mobile Netw. Appl.*, pp. 1–15, Sep. 2023, doi: 10.1007/s11036-023-02158-y.
- [10] M. El Khatib, S. Hamidi, I. Al Ameeri, H. Al Zaabi, and R. Al Marqab, "Digital disruption and big data in healthcare-opportunities and challenges," *ClinicoEcon. Outcomes Res.*, vol. 14, 563–574, Aug. 2022.
- [11] E. J. Thompson et al., "Long COVID burden and risk factors in 10 UK longitudinal studies and electronic health records," *Nat. Commun.*, vol. 13, no. 1, p. 3528, 2022.
- [12] S. Khasim and S. S. Basha, "An improved fast and secure CAMEL based authenticated key in smart health care system," *Cloud Comput. Data Sci.*, vol. 3, no. 2, pp. 77–91, 2022.
- [13] I. Yaqoob, K. Salah, R. Jayaraman, and Y. Al-Hammadi, "Blockchain for healthcare data management: Opportunities, challenges, and future recommendations," *Neural Comput. Appl.*, vol. 34, pp. 11475–11490, Jan. 2021.
- [14] J. Xu et al., "Healthchain: A blockchain-based privacy preserving scheme for large-scale health data," *IEEE Internet Things J.*, vol. 6, no. 5, pp. 8770–8781, Oct. 2019.
- [15] J. Chen, H. Xiao, M. Hu, and C.-M. Chen, "A blockchain-based signature exchange protocol for metaverse," *Future Gener. Comput. Syst.*, vol. 142, pp. 237–247, May 2023.
- [16] Q. Mei, H. Xiong, Y.-C. Chen, and C.-M. Chen, "Blockchain-enabled privacy-preserving authentication mechanism for transportation CPS with cloud-edge computing," *IEEE Trans. Eng. Manag.*, early access, Apr. 14, 2022, doi: 10.1109/TEM.2022.3159311.
- [17] S. Zhang, L. Wang, and H. Xiong, "Chaintegrity: Blockchain-enabled large-scale e-voting system with robustness and universal verifiability," *Int. J. Inf. Secur.*, vol. 19, pp. 323–341, Jun. 2020.
- [18] R. H. Taylor, "A perspective on medical robotics," *Proc. IEEE*, vol. 94, no. 9, pp. 1652–1664, Sep. 2006.
- [19] Z.-Y. Wu, Y.-C. Lee, F. Lai, H.-C. Lee, and Y. Chung, "A secure authentication scheme for telecare medicine information systems," *J. Med. Syst.*, vol. 36, no. 3, pp. 1529–1535, 2012.
- [20] H. Debiao, C. Jianhua, and Z. Rui, "A more secure authentication scheme for telecare medicine information systems," *J. Med. Syst.*, vol. 36, no. 3, pp. 1989–1995, 2012.
- [21] J. Wei, X. Hu, and W. Liu, "An improved authentication scheme for telecare medicine information systems," *J. Med. Syst.*, vol. 36, no. 6, pp. 3597–3604, 2012.
- [22] Z. Zhu, "An efficient authentication scheme for telecare medicine information systems," *J. Med. Syst.*, vol. 36, no. 6, pp. 3833–3838, 2012.
- [23] X. Jia, D. He, N. Kumar, and K.-K. R. Choo, "Authenticated key agreement scheme for fog-driven IoT healthcare system," *Wireless Netw.*, vol. 25, pp. 4737–4750, Nov. 2019.
- [24] T.-Y. Wu, T. Wang, Y.-Q. Lee, W. Zheng, S. Kumari, and S. Kumar, "Improved authenticated key agreement scheme for fog-driven IoT healthcare system," *Security Commun. Netw.*, vol. 2021, pp. 1–16, Jan. 2021.
- [25] C.-M. Chen, Y. Huang, K.-H. Wang, S. Kumari, and M.-E. Wu, "A secure authenticated and key exchange scheme for fog computing," *Enterprise Inf. Syst.*, vol. 15, no. 9, pp. 1200–1215, 2021.
- [26] S. Shamshad, M. S. Obaidat, U. Shamshad, S. Noor, and K. Mahmood, "On the security of authenticated key agreement scheme for fog-driven IoT healthcare system," in *Proc. Int. Conf. Artif. Intell. Smart Syst. (ICAIS)*, 2021, pp. 1760–1765.
- [27] C.-L. Chen, T.-T. Yang, M.-L. Chiang, and T.-F. Shih, "A privacy authentication scheme based on cloud for medical environment," *J. Med. Syst.*, vol. 38, pp. 1–16, Oct. 2014.
- [28] S.-Y. Chiou, Z. Ying, and J. Liu, "Improvement of a privacy authentication scheme based on cloud for medical environment," *J. Med. Syst.*, vol. 40, no. 4, pp. 1–15, 2016.

- [29] P. Mohit, R. Amin, A. Karati, G. Biswas, and M. K. Khan, "A standard mutual authentication protocol for cloud computing based health care system," *J. Med. Syst.*, vol. 41, pp. 1–13, Feb. 2017.
- [30] Q. Jiang, M. K. Khan, X. Lu, J. Ma, and D. He, "A privacy preserving three-factor authentication protocol for e-health clouds," *J. Supercomput.*, vol. 72, pp. 3826–3849, Jan. 2016.
- [31] F. Wu, L. Xu, S. Kumari, and X. Li, "A novel and provably secure biometrics-based three-factor remote authentication scheme for mobile client-server networks," *Comput. Elect. Eng.*, vol. 45, pp. 274–285, Jul. 2015.
- [32] Y. Chen and J. Chen, "A secure three-factor-based authentication with key agreement protocol for e-health clouds," *J. Supercomput.*, vol. 77, pp. 3359–3380, Apr. 2021.
- [33] M. Kumar and S. Chand, "A lightweight cloud-assisted identity-based anonymous authentication and key agreement protocol for secure wireless body area network," *IEEE Syst. J.*, vol. 15, no. 2, pp. 2779–2786, Jun. 2021.
- [34] B. A. Alzahrani, "Secure and efficient cloud-based IoT authenticated key agreement scheme for e-health wireless sensor networks," *Arab. J. Sci. Eng.*, vol. 46, no. 4, pp. 3017–3032, 2021.
- [35] N. Garg, M. Wazid, A. K. Das, D. P. Singh, J. J. Rodrigues, and Y. Park, "BAKMP-IoMT: Design of blockchain enabled authenticated key management protocol for Internet of Medical Things deployment," *IEEE Access*, vol. 8, pp. 95956–95977, 2020.
- [36] W. Wang et al., "Blockchain and PUF-based lightweight authentication protocol for wireless medical sensor networks," *IEEE Internet Things J.*, vol. 9, no. 11, pp. 8883–8891, Jun. 2022.
- [37] S. Yu and Y. Park, "A robust authentication protocol for wireless medical sensor networks using blockchain and physically unclonable functions," *IEEE Internet Things J.*, vol. 9, no. 20, pp. 20214–20228, Oct. 2022.
- [38] H. Xiong et al., "On the design of blockchain-based ECDSA with fault-tolerant batch verification protocol for blockchain-enabled IoMT," *IEEE J. Biomed. Health Inform.*, vol. 26, no. 5, pp. 1977–1986, May 2022.
- [39] M. Wazid, A. K. Das, and S. Shetty, "BSFR-SH: Blockchain-enabled security framework against ransomware attacks for smart healthcare," *IEEE Trans. Consum. Electron.*, vol. 69, no. 1, pp. 18–28, Feb. 2023.
- [40] X. Xiang, J. Cao, and W. Fan, "Decentralized authentication and access control protocol for blockchain-based e-health systems," *J. Netw. Comput. Appl.*, vol. 207, Nov. 2022, Art. no. 103512.
- [41] S. Son, J. Lee, M. Kim, S. Yu, A. K. Das, and Y. Park, "Design of secure authentication protocol for cloud-assisted telecare medical information system using blockchain," *IEEE Access*, vol. 8, pp. 192177–192191, 2020.
- [42] S. Itto, A. A. Khan, V. Kumar, A. Alkhayyat, M. Ahmad, and J. Srinivas, "CKMIB: Construction of key agreement protocol for cloud medical infrastructure using blockchain," *IEEE Access*, vol. 10, pp. 67787–67801, 2022.
- [43] L. Zhang et al., "BDSS: Blockchain-based data sharing scheme with fine-grained access control and permission revocation in medical environment," *KSII Trans. Internet Inf. Syst.*, vol. 16, no. 5, pp. 1634–1652, 2022.
- [44] P. Shrivastav and M. Sadasivan, "Blockchain-based system for secure data sharing in cloud using machine learning: Current researches and challenges," in *Proc. Int. Conf. Innov. Data Commun. Technol. Appl. (ICIDCA)*, 2023, pp. 1078–1084.
- [45] D. Dolev and A. Yao, "On the security of public key protocols," *IEEE Trans. Inf. Theory*, vol. 29, no. 2, pp. 198–208, Mar. 1983, doi: [10.1109/TIT.1983.1056650](https://doi.org/10.1109/TIT.1983.1056650).
- [46] R. Canetti and H. Krawczyk, "Universally composable notions of key exchange and secure channels," in *Proc. Int. Conf. Theory Appl. Cryptogr. Techn.*, 2002, pp. 337–351.
- [47] H. Zhang, J. Wang, and Y. Ding, "Blockchain-based decentralized and secure keyless signature scheme for smart grid," *Energy*, vol. 180, pp. 955–967, Aug. 2019.
- [48] T.-Y. Wu, L. Wang, X. Guo, Y.-C. Chen, and S.-C. Chu, "SAKAP: SGX-based authentication key agreement protocol in IoT-enabled cloud computing," *Sustainability*, vol. 14, no. 17, 2022, Art. no. 11054.
- [49] R. Canetti, O. Goldreich, and S. Halevi, "The random oracle methodology, revisited," *J. ACM*, vol. 51, no. 4, pp. 557–594, 2004.
- [50] M. Abdalla, P.-A. Fouque, and D. Pointcheval, "Password-based authenticated key exchange in the three-party setting," in *Proc. Int. Workshop Public Key Cryptogr.*, 2005, pp. 65–84.
- [51] T.-Y. Wu, L. Yang, J.-N. Luo, and J. Ming-Tai Wu, "A provably secure authentication and key agreement protocol in cloud-based smart healthcare environments," *Secur. Commun. Netw.*, vol. 2021, pp. 1–15, Sep. 2021.
- [52] D. Wang, H. Cheng, P. Wang, X. Huang, and G. Jian, "Zipf's law in passwords," *IEEE Trans. Inf. Forensics Security*, vol. 12, pp. 2776–2791, 2017.
- [53] S. Heron, "Advanced encryption standard (AES)," *Netw. Secur.*, vol. 2009, no. 12, pp. 8–12, 2009.
- [54] M. Ge, S. Kumari, and C.-M. Chen, "AuthPFS: A method to verify perfect forward secrecy in authentication protocols," *J. Netw. Intell.*, vol. 7, no. 3, pp. 734–750, 2022.
- [55] M. Ma, D. He, H. Wang, N. Kumar, and K.-K. R. Choo, "An efficient and provably secure authenticated key agreement protocol for fog-based vehicular Ad-Hoc networks," *IEEE Internet Things J.*, vol. 6, no. 5, pp. 8065–8075, Oct. 2019.
- [56] T.-Y. Wu, X. Guo, L. Yang, Q. Meng, and C.-M. Chen, "A lightweight authenticated key agreement protocol using fog nodes in social Internet of Vehicles," *Mobile Inf. Syst.*, vol. 2021, pp. 1–14, Nov. 2021.
- [57] X. Li, T. Chen, Q. Cheng, and J. Ma, "An efficient and authenticated key establishment scheme based on fog computing for healthcare system," *Front. Comput. Sci.*, vol. 16, pp. 1–12, Aug. 2022.
- [58] S. A. Eftekhari, M. Nikooghadam, and M. Rafighi, "Security-enhanced three-party pairwise secret key agreement protocol for fog-based vehicular Ad-Hoc communications," *Veh. Commun.*, vol. 28, Apr. 2021, Art. no. 100306.
- [59] S. Shamshad, M. S. Obaidat, M. A. Saleem, U. Shamshad, and K. Mahmood, "Security analysis on an efficient and provably secure authenticated key agreement protocol for fog-based vehicular ad-hoc networks," in *Proc. Int. Conf. Artif. Intell. Smart Syst. (ICAIS)*, 2021, pp. 1754–1759.
- [60] Z. Li, Q. Miao, S. A. Chaudhry, and C.-M. Chen, "A provably secure and lightweight mutual authentication protocol in fog-enabled social Internet of Vehicles," *Int. J. Distrib. Sens. Netw.*, vol. 18, no. 6, 2022, Art. no. 15501329221104332.
- [61] "SawtoothMasters." Accessed: May 1, 2024. [Online]. Available: <https://github.com/wodemaya/SawtoothMasters>